

国内图书分类号：TP391.9

密级：公开

国际图书分类号：621.38

西南交通大学 研究生学位论文

基于物理的大气渲染

年 级 二零一八级
姓 名 陈凯鑫
申请学位级别 工学硕士
专 业 控制科学与工程
指 导 老 师 苏虎

二零二一年四月七日

Classified Index: TP391.9

U.D.C: 621.38

Southwest Jiaotong University
Master Degree Thesis

PHYSICALLY BASED ATMOSPHERE
RENDERING

Grade: 2018

Candidate: Kaixin Chen

Academic Degree Applied for: Master of Science in Engineering

Specialty: Control Science and Engineering

Supervisor: Hu Su

Apr.7, 2021

西南交通大学硕士学位论文主要工作（贡献）声明

本人在学位论文中所做的主要工作或贡献如下：

1. 对当前国内外的渲染技术发展状况进行了分析，重点阐述了体渲染技术的原理以及应用场景，对大气渲染中的各项关键技术进行了阐述。
2. 以辐射度量学理论为基础，对光线在大气以及云层中的传播过程进行了建模，分析了光线在传播过程中的能量变化，并根据大气以及云层的特点，对大气散射效果以及动态云层效果提出了不同的渲染方法。
3. 在完成对大气散射效果和动态云层效果的渲染的基础上，将这两者进行整合，得到了完整的大气渲染效果，可以模拟不同污染程度以及不同太阳高度下的天空效果，并对算法进行了优化，满足实时渲染的需求。

本人郑重声明：所呈交的学位论文，是在导师指导下独立进行研究工作所得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究做出贡献的个人和集体，均已在文中作了明确说明。本人完全了解违反上述声明所引起的一切法律责任将由本人承担。

学位论文作者签名：陈凯舒
日期：2021.5.16

摘 要

大气层是地球环境的重要组成部分,在进行室外场景的渲染时,大气的渲染质量对整个场景最终呈现的视觉效果起到了重要作用。一种方法是绘制静态的,包括天空以及云层的纹理,使用纹理映射技术对大气进行渲染。但这种静态的天空效果在真实感上较为欠缺,并且无法随太阳高度角的变化而动态变化。因此,为了提升室外场景渲染的真实感,需要一种对地球大气进行准确渲染的渲染方法。

本文实现了一种基于物理的大气渲染方法,该方法针对 GPU 并行计算架构的进行设计,通过对光线在大气传播过程中发生的能量改变进行建模,以及对体渲染理论的应用,计算光线在大气中经过散射后进入视点的能量,从而实现准确的大气渲染。同时,通过对大气层内云层的特性进行研究,该方法使用噪声纹理对云层形状与分布进行模拟,并使用光线步进技术计算云层的光照,得到了较好的云层渲染效果。该方法还通过对历史渲染数据的复用,提高云层渲染的效率,满足实时渲染的需求。最后,该方法将大气散射效果以及动态云层效果相结合,实现了动态的大气层渲染效果,提升了室外场景的真实感。

在室外场景中整合本文实现的渲染方法时,不仅可以实现随太阳高度角动态变化的大气效果,为场景中物体提供动态的环境光照,还可以通过分别对大气层以及云层参数的调整,模拟不同污染程度下的视觉效果,改变云层的形状与分布。该方法可以满足实时渲染的需求,提升渲染系统的真实感。

关键词: 基于物理的渲染; 实时渲染; 光线步进; 大气散射; 体积云

Abstract

The atmosphere is an important component of the Earth's environment, and when rendering outdoor scenes, the quality of the atmosphere rendering plays an important role in the final visual effect of the entire scene. One way to render the atmosphere is to map the skybox texture to the entire sky using texture mapping techniques. However, this static sky effect is lacking in realism and cannot change dynamically with the change of solar zenith angle. Therefore, to improve the realism of outdoor scene rendering, a rendering method for accurate rendering of the earth's atmosphere is needed.

In this thesis, a physically based atmospheric rendering method is implemented. The method is designed for GPU parallel computing architecture, which calculates the energy of light entering the viewpoint after scattering in the atmosphere by modeling the energy change of light propagation in the atmosphere and applying the volumetric rendering theory to achieve an accurate atmospheric rendering. Meanwhile, by studying the characteristics of clouds within the atmosphere, this method uses noise textures to simulate cloud shapes and distributions, and uses light stepping techniques to calculate the lighting of clouds to obtain a better quality of cloud rendering. This method also improves the efficiency of cloud rendering by reusing historical rendering results to meet the requirements of real-time rendering. Finally, this method combines the atmospheric scattering effect and the dynamic volumetric clouds to achieve a dynamic atmospheric rendering effect and improve the sense of reality of outdoor scenes.

When integrating the method implemented in this paper in outdoor scenes rendering, it is not only possible to achieve atmospheric effects that dynamically change with the solar zenith angle and provide dynamic ambient lighting for objects in the scene, but also to simulate the visual effects under different pollution levels and change the shape and distribution of clouds by respectively adjusting atmospheric and cloud parameters. This method can meet the requirements of real-time rendering and improve the sense of reality of the rendering system.

Keywords: Physically based rendering; Real-time rendering; Ray marching; Atmosphere scattering; Volumetric clouds

目 录

第 1 章	绪论	1
1.1	研究背景与意义	1
1.2	研究现状	1
1.3	主要研究工作	3
1.4	本文组织结构	4
第 2 章	体渲染原理	5
2.1	色彩空间与辐射度量学简介	5
2.1.1	光谱到色彩空间的变换	5
2.1.2	辐射度量学简介	7
2.1.3	色彩空间与辐射度量学的联系	8
2.2	光在介质中传播时的能量变化	8
2.2.1	消光	9
2.2.2	内散射和发射	9
2.3	体渲染方程	10
2.4	求解体渲染方程使用的方法	11
2.5	本章小结	11
第 3 章	大气渲染的算法	12
3.1	体渲染方程分析	12
3.2	透射率的预计算算法	12
3.2.1	消光系数的确定	13
3.2.2	透射率的计算	14
3.3	内散射的预计算算法	15
3.3.1	直接散射的预计算算法	15
3.3.2	间接散射的预计算算法	17
3.4	本章小结	19
第 4 章	体积云的渲染技术	20
4.1	噪声纹理在体积云渲染中的应用	20
4.1.1	Perlin 噪声	20

4.1.2	Worley 噪声	22
4.1.3	分形噪声	23
4.2	云层的生成	25
4.3	体积云的光照模型	27
4.4	体积云渲染的优化	29
4.5	本章小结	31
第 5 章	基于物理的大气实时渲染技术的设计与实现	32
5.1	渲染系统的总体架构	32
5.2	基于 DirectX 12 的渲染流程.....	33
5.3	大气渲染的实现	36
5.3.1	预计算纹理的生成	37
5.3.2	物体的环境光照计算	43
5.3.3	天空颜色的计算	44
5.4	体积云渲染的实现	45
5.4.1	体积云的体渲染技术	46
5.4.2	体积云渲染所使用的纹理的生成	48
5.4.3	体积云密度的采样	49
5.4.4	体积云的光照计算	52
5.4.5	体积云渲染的优化	56
5.5	渲染系统测试与性能分析	58
5.5.1	测试环境	58
5.5.2	大气渲染的效果及效率	59
5.5.3	增加云层渲染后的效果及效率	61
5.5.4	与其他方法的比较	65
5.6	本章小结	67
总结与展望		68
致 谢		69
参考文献		70
攻读硕士学位期间发表的论文及科研成果		74

第1章 绪论

1.1 研究背景与意义

大气散射是自然界中非常普遍而又十分重要的物理现象。由于大气散射的存在,天空可以在太阳的照射下,在不同时间呈现不同的颜色;当空气较为潮湿时,大气散射又会使得远景变得模糊,空气也会被大气照亮,产生体积光的效果。这些常见的效果不仅有渲染气氛,烘托氛围的效果,也对营造真实感有着非常重要的作用。在构建虚拟场景时,如果不考虑大气散射效应,会让空气显得非常“干净”,从而让人产生不真实感。

在计算机图形学中,传统的绘制天空的方法主要为使用立方体映射,将天空盒的纹理映射到虚拟场景的天空上。这种方法简单快速,但是无法模拟天空随时间变化,动态的云层,体积光等效果。同时,传统的光照计算,将光看作一条射线,只对投射到物体上的光线进行计算,得到物体的颜色,而忽视了光线在传播过程中,照射到空气分子上,照亮空气的过程。因此,整个场景缺乏真实的效果,其本质是虚拟环境中没有对光线传播进行符合物理规则的模拟。

随着计算机处理器的性能提升,尤其是图形处理器的计算能力的提升,使得从物理的角度,对光线与物体的交互进行计算模拟成为可能。目前已经有基于物理的渲染技术(Physically-based Rendering)被广泛应用与各种模拟器与游戏中。因此,为了解决虚拟场景中大气效果不真实的问题,需要研究实时渲染速度下大气的渲染问题,对光线在大气中的各种物理效果进行建模,得到真实的渲染效果。本课题选择虚拟场景中的大气作为渲染的研究对象,以实现在实时速率下的大气效果渲染。

1.2 研究现状

大气的渲染问题在计算机图形学中属于真实感图形学的研究范畴。使用传统的真实感图形学渲染方法,之所以很难展示大气散射的效果,是因为传统的渲染方式假定了物体一定是具有具体的形状的,所有的图形学计算只发生在光线不能穿透的材质的表面。这对于不透明的实体来说,具有非常高的效率,但是对于透明的物体来说,则很难正确进行计算。随着渲染理论与技术的进步,图形学界陆续开始对大气光照模型进行研究,相关的渲染模型被相继提出。第一个基于物理的大气渲染模型由 Nishita 等人于 1993 年提出,从太空中的视角对地球及其大气进行了渲染,对太阳光在大气中的单次散射进行计算^[1]。之后,学术界提出的大气光照模型可以大致分为解析模型和迭代模型

两种^[2]。解析模型将天空的外观参数化,并对相应的参数进行测量,得到最终的计算结果。例如 A. J. Preetham 于 1999 年提出了一种方法,对空气的浑浊度,即大气中光线发生米氏散射的表相,太阳光的亮度,视角方向以及太阳方向等参数进行测量,并根据参数表计算出相应的渲染结果^[3]。然而,这些模型受限于对参数的测量,只能实现从地面视角出发的大气渲染,也不能通过适当改变参数,正确地渲染地外行星的大气。

迭代模型则依赖于一个基本假设,即地球以及大气层是一个完美的球形,且大气是一个均匀的介质^[4]。基于这个假设,就可以使用光线步进对散射光进行计算,计算的结果通常会被保存在查找表中,避免重复的光线步进计算。

O'Neil 于 2005 年提出了一种使用光线步进对大气层边界与地面之间的散射进行计算的方法。对于较为低频的瑞利散射和米氏散射,在顶点着色器中进行计算,而对于高频的相函数则在像素着色器中进行计算。并可以将结果存储在纹理中,对一些计算进行加速,得到了不错的效果^[5]。

Eric Bruneton 和 Fabrice Neyret 于 2008 年提出了基于预计算的大气渲染算法,在实时渲染阶段前对大气渲染体渲染方程中的一些项进行预计算,并将预计算结果存储在查找表中。在实时渲染阶段对查找表进行查找,除了可以得到单次散射辐射率的计算结果外,还可以得到多次散射辐射率的近似解,性能上也十分优异^[6]。2017 年, Eric Bruneton 还对代码进行了优化,增加了可读性和健壮性。

Oskar Elek 于 2009 年对 Eric Bruneton 和 Fabrice Neyret 的方法进行了优化,将其中使用 4 个维度变量计算多次散射辐射率的方法进行了优化,仅使用 3 个维度的变量就对多次散射辐射率进行了计算,并对查找表进行了优化^[7]。

Egor Yusov 也与 2013 年提出了一种利用 3 个维度的变量对多重散射进行计算的方法,并对性能进行了优化,减少了查找表的体积以及更新的时长^[8]。但该方案无法在大气层内透射阴影,因此无法渲染出光柱的效果。

Sébastien Hillaire 在 2016 年对 Bruneton, Elek 和 Yusov 的方法进行了改进,改进后的方法可以让美术人员通过对大气参数的调整,制作出不同视觉效果的天空。该方法在 Electronic Arts 公司的 Frostbite 引擎中得以实现,并在诸如 Need for Speed, Mirror's Edge Catalyst 等游戏中应用^[2]。

2020 年, Sébastien Hillaire 又对方法进行了优化,并在包括手机等移动平台在内的多个平台上实现的相应的方法^[9]。

由于云层形状的复杂性与易变性,云层的渲染比大气的渲染更为复杂。在很长的一

段时间内, 渲染云层的方法是利用透明度, 将天空的颜色与绘有云层的纹理颜色相混合, 得到静态的云层效果。2014 年, Keith Guerrette 提出了一种方法, 可以给静态云层赋予随着风向运动的错觉, 但是这种方法并不能提供云层以动态的光照^[10]。

另一种渲染云层的方法是将云层看作由许多微粒组成的系统, 并对每个微粒进行计算以及渲染。Egor Yusov 于 2014 年提出了一种方法, 利用查找表记录渲染画面上, 每个云层微粒的散射辐射率和透射率, 从而对视角范围内的云层进行渲染, 得到较好的效果^[11]。

Bouthors 等人于 2008 年使用网格描述云层的形状, 利用 Hypertexture 为云层的表面添加高频的细节, 并利用光线步进计算云层的辐射率。对于单次散射的辐射率, 该方法使用解析模型进行计算, 多次散射的辐射率则使用布置在云层内部的光照探针进行离线计算, 并在实时渲染阶段对计算结果进行读取, 得到高质量的渲染结果^[12]。

近年来, 较为流行的方式是使用光线步进的方法, 将云层作为光线传播路径中的介质进行考虑, 计算云层最终的辐射率。Julian Fong 等人在 2017 年使用路径追踪的方式对云层进行渲染, 得到了很好的效果, 但由于渲染时间较长, 主要应用于离线渲染^[13]。Andrew Schneider 在 2017 年提出了使用 Perlin 噪声, Worley 噪声对云层进行建模, 渲染的方法。并使用云层的分布纹理, 确定云层的分布的具体位置, 便于大型场景中对云层进行编辑, 并应用于游戏 Horizon: Zero Dawn 中^[14]。2019 年, Fabian Bauer 等人对 Schneider 的方法进行的优化, 对更多类型的云进行了建模, 并对实现了云层与天气系统的整合, 并将该方法应用于游戏 Red Dead Redemption 2 中^[15]。

1.3 主要研究工作

设计一套基于 GPU 实现的大气渲染算法, 实现实时条件下, 大气以及云层的渲染。主要从以下几个方面展开:

1. 对基于物理的渲染方法进行研究

从辐射度量学的角度出发, 使用数学语言对光线的能量进行描述, 并且利用色彩空间理论, 建立光线能量与计算机显示颜色之间的联系, 为基于物理的渲染打下基础。

2. 对光线在大气中传播时发生的能量变化进行建模

利用辐射度量学的理论, 对光线在大气中传播时发生的各种物理现象进行建模, 描述能量的具体变化。并从简化体渲染方程入手, 结合光照理论, 对渲染方程的各个部分进行分析, 便于利用 GPU 实现大气的渲染。

3. 对云的形状特征以及光线在云层内部传播时发生的能量变化进行建模

对云层的形状特征进行观察,利用噪声纹理,通过程序式生成的方式,生成具有云层形状特定的云层。同时根据云层的特点,对光线在云层内部的能量变化进行建模,求解相应的近似解。

4. 利用 DirectX 12 渲染 API,实现对大气以及云层的渲染

利用 DirectX 12 对渲染管线的高度定制能力,以及计算着色器的特点,利用计算着色器,对大气渲染中的关键参数进行预计算,加速实时渲染计算的计算。并且,利用光线步进技术,对云层的渲染,并对性能进行优化。

5. 将大气渲染与云层渲染相结合,在场景中渲染基于物理的天空

利用基于物理的渲染技术,完成对大气以及云层的渲染后,将大气效果和体积云效果相结合,给出在不同光线入射角度时,正确着色的天空。

1.4 本文组织结构

本文各章节安排如下:

第 1 章,对本文的研究背景以及意义进行介绍,并阐述课题在国内外的研究现状,以及本文的主要研究工作和组织结构。

第 2 章,介绍色彩空间以及辐射度量学理论,阐述对光线的能量进行度量以及计算的方法,并建立能量与计算机显示的颜色之间的联系。并对体渲染方程进行分析,给出后文求解体渲染方程的思路。

第 3 章,对光线在大气传播过程中,发生的各种物理现象进行建模,给出对应的能量变化的描述。并根据体渲染方程以及大气层的特点,给出通过对体渲染方程的一些项进行预计算,加速体渲染方程的方法。

第 4 章,研究噪声技术在模拟云层形状中的应用,阐述利用噪声纹理,生成云层形状、空间排布的方法。并对云层的光照模型进行分析,提出求解云层体渲染方程近似解的方法。并利用云层的特点,提出对云层渲染进行优化的算法。

第 5 章,对渲染系统的总体架构,渲染管线的结构,以及大气渲染和云层渲染的具体流程进行介绍,并对体积云的渲染优化方法进行介绍,将最终的渲染结果进行展示。

最后对本文的工作进行总结以及对研究成果的继续完善进行展望。

第2章 体渲染原理

2.1 色彩空间与辐射度量学简介

在进行计算机渲染时,为了能够准确地描述光线与物体交互的物理过程,并对这些物理过程中的能量变化进行采样、计算,最终得到正确着色的图像,需要对计算机的色彩表示,光谱到色彩空间的转换,以及电磁辐射在空间中的传播,即辐射度量学,等计算机图形学中的一些概念进行简单的介绍。

2.1.1 光谱到色彩空间的变换

人类眼睛有对于短(S, 420-440nm)、中(M, 530-540nm)和长(L, 560-580nm)波长的光感受器,称为视锥细胞。因此,根据三种视锥细胞的刺激比例,便能描述任一种颜色的感觉,此称为 LMS 空间^[16]。根据 LMS 空间理论以及格拉斯曼定律:人类对色彩的感知大约是线性的^[17],可以通过指定三种基色,通过混色处理表现任意第四种颜色。在 19 世纪 20 年代,W. David Wright 和 John Guild 独立进行了一系列人类视觉实验,使用标准波长为 700 nm(红色)、546.1 nm(绿色)和 435.8 nm(蓝色)的三种单色原色,对一些不同的观察者进行了色彩合成实验,记录了合成各种波长的可见光的颜色所需的三种基色的强度,得到了色匹配函数。他们的实验结果被国际照明委员会(Commission Internationale de l'éclairage, CIE)整理成了 CIE1931-2deg-XYZ 色彩空间^[16]。该色彩空间的色度图如图 2-1 所示。需要说明的是,色彩空间是一个三维空间,该色度图是色彩空间在 xy 平面的一个投影。

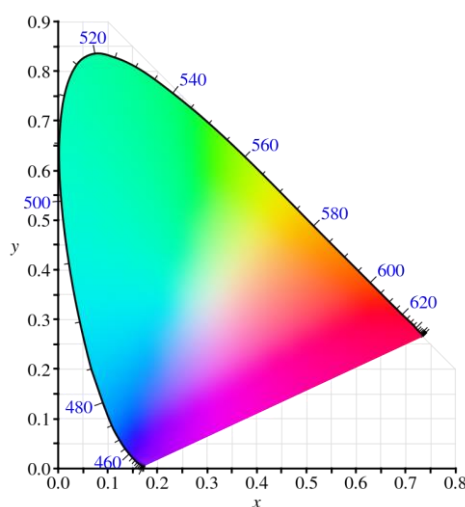
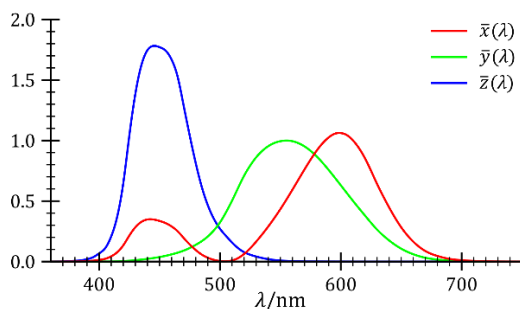


图 2-1 CIE1931-2deg-XYZ 色彩空间色度图^[16]

该色彩空间对应的色匹配函数 $\bar{x}(\lambda)$, $\bar{y}(\lambda)$, $\bar{z}(\lambda)$ 如图 2-2 所示:

图 2-2 CIE1931-2deg-XYZ 色彩空间的色匹配函数^[16]

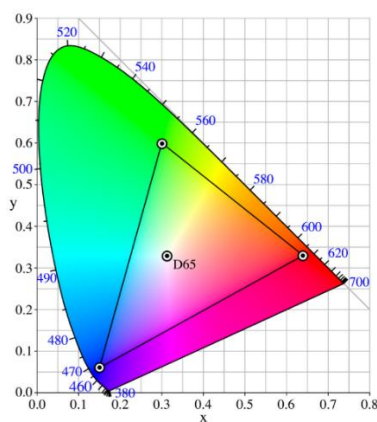
对于任意给出频谱功率分布 $I(\lambda)$ 的光，其颜色在 CIE1931-2deg-XYZ 色彩空间的坐标 (X,Y,Z) 可以由公式（2-1）计算得到。

$$\begin{aligned} X &= \int_0^{\infty} I(\lambda) \bar{x}(\lambda) d\lambda \\ Y &= \int_0^{\infty} I(\lambda) \bar{y}(\lambda) d\lambda \\ Z &= \int_0^{\infty} I(\lambda) \bar{z}(\lambda) d\lambda \end{aligned} \quad (2-1)$$

由于 CIE1931-2deg-XYZ 色彩空间对 W. David Wright 等人的实验数据进行了线性变换，色彩空间的基色并不是真实存在的颜色。因此，为了得到实际使用的 RGB 色彩空间中的坐标，需要将 (X,Y,Z) 坐标进行一个线性变换，得到对应 RGB 色彩空间下的坐标值。本文使用 sRGB(standard Red Green Blue)色彩空间， (R,G,B) 坐标的计算可由公式（2-2）计算得到^[18]。

$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 3.2410 & 1.5374 & 0.4986 \\ -0.9692 & 1.8760 & 0.0416 \\ 0.556 & -0.2040 & 1.0570 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix} \quad (2-2)$$

sRGB 色彩空间在 CIE1931-2deg-XYZ 色彩空间中的区域如图 2-3 所示：

图 2-3 sRGB 色彩空间^[18]

图中三角形围起的区域为 sRGB 色彩空间所能表示的所有颜色。D65 点所在的位置

为 sRGB 色彩空间中白色的位置。

2.1.2 辐射度量学简介

辐射度量学为光线的传播和反射提供了数学的描述,是计算机图形学中常用的理论,也是基于物理的渲染算法的基础。本节将介绍辐射度量学中的一些基本概念。

2.1.2.1 能量

光具有波粒二象性,当计算光的能量时,可以计算光子所携带的能量 Q , 由公式 (2-3) 给出^[19], 单位为 J。其中, h 为普朗克常量, c 为光速, λ 为光的波长。

$$Q = \frac{hc}{\lambda} \quad (2-3)$$

2.1.2.2 辐射通量

辐射通量(Radiant Flux)描述的是单位时间内通过某一平面的能量的总量,即光的功率。辐射通量 Φ 可由公式 (2-4) 计算得到, 单位为 J/s, 或者 W^[20]。

$$\Phi = \frac{dQ}{dt} \quad (2-4)$$

2.1.2.3 辐射强度

辐射强度(Radiant Intensity)描述的是光线在单位立体角上的功率。立体角是弧度在三维空间内的延伸,即球面上某一区域的面积除以球体半径的平方,单位为 sr。如图 2-4 所示。

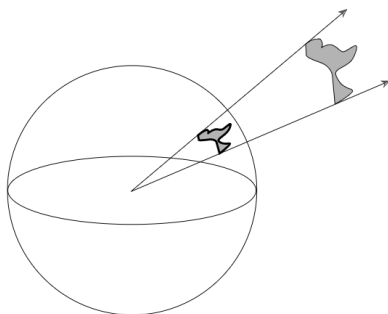


图 2-4 立体角示意图^[20]

辐射强度 I 描述了光在方向上的分布, 但仅对点光源有意义。可由公式 (2-5) 计算得到, 单位为 W/sr^[20]。

$$I = \frac{d\Phi}{d\omega} \quad (2-5)$$

2.1.2.4 辐照度

辐照度(Irradiance)描述的是单位面积上光的功率。对于平面上一点 p , 该点的辐照度 $E(p)$ 可由公式 (2-6) 计算得到, 单位为 W/m²^[20]。

$$E(p) = \frac{d\Phi(p)}{dA} \quad (2-6)$$

2.1.2.5 辐射率

辐照度给出了光在单位面积上的功率分布,但由于光具有方向性,辐照度无法给出光线功率在方向上的分布,因此需要引入辐射率(Radiance)的概念。辐射率 L 定义了平面上一点 p , 沿方向 ω 的功率, 可由公式 (2-7) 计算得到, 单位为 $\text{W}/(\text{sr} \cdot \text{m}^2)^{[20]}$ 。

$$L = \frac{d\Phi}{d\omega dA^\perp} \quad (2-7)$$

其中 $dA^\perp$ 为 dA 投影到方向 ω 上的面积。如图 2-5 所示。辐射率的大小同时也决定了光线的亮度大小。

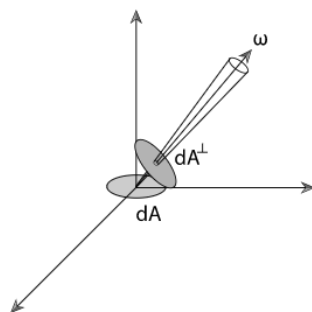


图 2-5 辐射率示意图^[20]

2.1.3 色彩空间与辐射度量学的联系

CIE 在制定色彩标准时也考虑到了亮度(Luminance)对颜色的影响, CIE1931-2deg-XYZ 色彩空间也建立了与辐射率之间的联系。人眼的对不同波长的相对感应灵敏度曲线 $V(\lambda)$ 与 CIE1931-2deg-XYZ 色彩空间对应的色匹配函数中的 $\bar{y}(\lambda)$ 成大致正比, 因此亮度值 Y 与辐射率 L 之间的关系可以由公式 (2-8) 表述^[20]。其中 683 为最大光功当量的值, 表示功率为 1W, 波长为 555.016nm 的光的亮度为 683 流明(lm)。

$$Y = 683 \int_{\lambda} L(\lambda) \bar{y}(\lambda) d\lambda \quad (2-8)$$

2.2 光在介质中传播时的能量变化

传统的基于反射理论的计算机图形学着色理论聚焦于光线在材质表面的反射以及能量变化, 并且假设光线在离开一个物体表面, 到达另一个物体表面的过程中, 辐射率没有发生变化 (或是使用简单的衰减模型进行衰减计算)。但实际上, 光并不在理想的真空中传播, 在空气中, 光子会和空气中的微粒发生碰撞, 发生能量的吸收, 或运动方向改变等现象, 这也是天空颜色的来源, 如果没有这些现象, 天空将会呈现于月球上天

空一样的黑色。在对大气, 云层等难以用表面模型去描述的物体进行渲染时, 就需要考虑到光在其中传播时能量发生的变化。本节将介绍光线传播过程中发生的物理现象, 并对其进行建模。

2.2.1 消光

消光(Extinction)是天文学中观测者用来描述被观测的天体发射的电磁辐射被路途中的物质, 如气体和尘埃吸收(Absorption)和外散射(Out-Scattering)的过程^[21]。在物理学中, 电磁辐射的吸收是指物质吸收光子能量并将其转化为自身内能的过程^[22]; 散射则指光子与微粒发生碰撞后, 改变了原先的运动方向, 向其他方向出射的过程。吸收和外散射带来的结果是, 光的辐射率沿着光的传播方向递减。设每一次发生吸收和外散射时, 光线损失的能量分别为 $1 - \sigma_a$ 和 $1 - \sigma_s$, 那么对于在点 x 处, 出射方向为 $\vec{\omega}$, 辐射率为 L 的光 $L(x \rightarrow \vec{\omega})$, 在 x 附近的辐射率变化可以由公式(2-9)表示^[23]:

$$(\vec{\omega} \cdot \nabla)L(x \rightarrow \vec{\omega}) = -(\sigma_a(x) + \sigma_s(x))L(x \rightarrow \vec{\omega}) \quad (2-9)$$

记 $\sigma_t(x) = \sigma_a(x) + \sigma_s(x)$, $\sigma_t(x)$ 为消光系数。通过对上式微分方程求解可得公式(2-10)^[23]:

$$L(x \leftarrow \vec{\omega}) = T_r(x' \leftrightarrow x)L(x' \rightarrow -\vec{\omega}) \quad (2-10)$$

其中, T_r 为透射率, 可以由公式(2-11)计算得到, 其中 $x' \leftrightarrow x$ 表示点 x 与 x' 之间的透射率只与 x 和 x' 之间的位置关系有关, 与方向无关。

$$T_r(x' \leftrightarrow x) = e^{-\tau(x' \leftrightarrow x)} \quad (2-11)$$

其中, τ 成为光学深度, 表示的是消光系数 σ_t 随传播路径的线积分, 如公式(2-12)所示^[23]:

$$\tau(x' \leftrightarrow x) = \int_0^d \sigma_t(x + t\vec{\omega})dt \quad (2-12)$$

2.2.2 内散射和发射

光线在传播的过程中除了发生能量的损失外, 还有可能发生能量的增加。增加的能量主要来自内散射(In-Scattering)和发射(Emission)。

发射是指, 当化学元素中处于激态的电子离开激态, 返回低能量的轨道时, 能量会以电磁辐射的形式被再辐射出来, 从而增加辐射率^[24]。例如火焰、氖灯就以这种形式发出可见光。由于在大气中, 没有能够在自然条件下发光的气体, 因此本文忽略这个因素。

内散射指当发生散射时,光子改变了运动方向,向 $\vec{\omega}'$ 方向出射。那么对于原先就沿 $\vec{\omega}'$ 方向传播的光线而言,由于接受了来自其它方向的散射光,它的辐射率增加了。对于散射的光线,可以使用相函数 $p(x, \vec{\omega}' \leftrightarrow \vec{\omega})$ 描述其出射反向的角度分布。对于不同种类的介质,光线沿各个方向反射的比例不同,如图 2-6 所示。

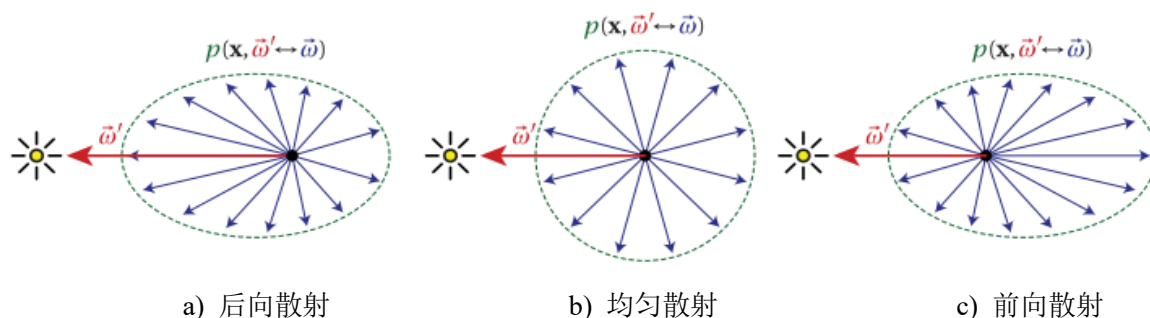


图 2-6 不同介质光线散射角度分布示意图^[23]

点 x 处周围,向 $\vec{\omega}$ 方向内散射的辐射率之和 $L_i(x \rightarrow \vec{\omega})$ 可以用公式(2-13)计算得到。其中 $p(x, \vec{\omega}' \leftrightarrow \vec{\omega})$ 为相函数, $L(x \leftarrow \vec{\omega}')$ 为以 $\vec{\omega}'$ 方向入射点 x 的光线的辐射率。该式沿着点 x 为球心的球面计算了所有方向入射光往 $\vec{\omega}$ 方向散射强度的积分。

$$L_i(x \rightarrow \vec{\omega}) = \int_{\Omega_{4\pi}} p(x, \vec{\omega}' \rightarrow \vec{\omega}) L(x \leftarrow \vec{\omega}') d\vec{\omega}' \quad (2-13)$$

最常用的相函数为 Henyey-Greenstein(HG)相函数。HG 散射函数常用来表示非均匀散射介质,如海洋,云朵,皮肤等的散射角度分布。HG 散射的公式如公式(2-14)所示。其中, θ 为入射方向和散射方向之间的夹角, $g \in [-1,1]$,表示介质向前或者向后散射的强度。 $g < 0$ 时,主要发生后向散射,如图 2-6a 所示; $g = 0$ 时,发生均匀散射,光线向各个方向散射的强度相等,如图 2-6b 所示;当 $g > 0$ 时,主要发生前向散射,如图 2-6c 所示^[25]。

$$p_{HG}(g, \theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g \cos \theta)^{1.5}} \quad (2-14)$$

2.3 体渲染方程

在介绍了光在传播过程中的各种物理现象后,设吸收系数为 σ_a ,散射系数为 σ_s ,就可以给出光的放射转移方程,如公式(2-15)所示^[26]。其中 $-(\sigma_a(x) + \sigma_s(x))L(x \rightarrow \vec{\omega})$ 为消光项, $\sigma_s(x)L_i(x \rightarrow \vec{\omega})$ 为内散射项。

$$(\vec{\omega} \cdot \nabla)L(x \rightarrow \vec{\omega}) = -(\sigma_a(x) + \sigma_s(x))L(x \rightarrow \vec{\omega}) + \sigma_s(x)L_i(x \rightarrow \vec{\omega}) \quad (2-15)$$

求解该微分方程后,可以得到体渲染方程,如公式(2-16)所示^[23]。

$$L(x \leftarrow \vec{\omega}) = T_r(x \leftrightarrow x_s)L(x_s \rightarrow -\vec{\omega}) + \int_0^s T_r(x \leftrightarrow x_t)\sigma_s(x_t)L_i(x_t \rightarrow -\vec{\omega})dt \quad (2-16)$$

其中, $L(x \leftarrow \vec{\omega})$ 为光以方向 $\vec{\omega}$ 到达点 x 处时的辐射率, $x_s = x + s\vec{\omega}$, $x_t = x + t\vec{\omega}$ 。 $L(x_s \rightarrow -\vec{\omega})$ 为视线终点表面反射的辐射率。 $T_r(x \leftrightarrow x')$ 描述的是光从 x 到 x' 处时, 辐射率的衰减, 即 x 与 x' 之间的透射率, $T_r(x \leftrightarrow x')$ 的计算由公式(2-11)给出。 $L_i(x_t \rightarrow -\vec{\omega})$ 为路径各点处内散射的辐射率, 计算由公式(2-13)给出。图 2-7 对体渲染方程中的各项进行了展示。

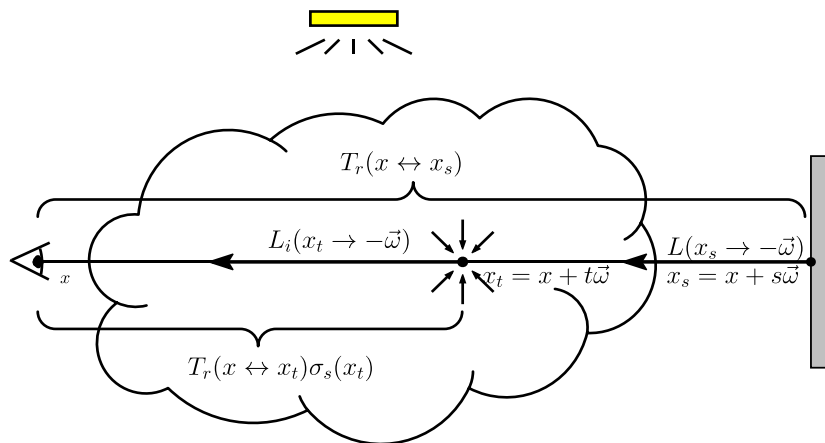


图 2-7 体渲染方程说明^[23]

2.4 求解体渲染方程使用的方法

对大气, 云层等非实体进行光照计算的时候, 就需要对体渲染方程进行求解。然而体渲染方程是一个非常复杂的积分式, 比基于微面元理论和双向反射分布函数的表面反射方程要复杂得多, 介质中一点的辐射率不仅取决于自身, 还需要计算介质中其它所有点的贡献。因此几乎不可能在实时的情况下求解。本课题的主要思路是, 对于密度较为均匀, 可以进行估算的大气层, 利用体渲染方程的一些性质, 拆解积分并进行预计算, 将计算结果存储在纹理中, 在实时渲染时直接读取, 减少计算量。对于密度随机的云层, 则从概率的角度对体渲染方程中的项进行估算, 得到近似的解。

2.5 本章小结

本章主要介绍的是本文所实现的大气渲染中所使用到的渲染模型的理论, 首先介绍了色彩空间, 建立了光的波长与计算机所使用的色彩之间的联系。之后从光度学和辐射度量学的角度, 描述了对光的能量进行度量, 计算的方法。最后介绍了光子与介质中微粒碰撞时发生的吸收和散射的现象, 并给出了对应的数学描述, 从而引出了体渲染方程, 并对本文求解体渲染方程所使用的方法进行了简单介绍。

第3章 大气渲染的算法

3.1 体渲染方程分析

由公式(2-16)可知,体渲染方程可以分解为三项数值的计算,分别为透射率 T_r ,内散射辐射率 L_i 和来自物体表面反射的辐射率 L 。其中物体表面反射的辐射率 L 可以在渲染物体时得到。对于透射率 T_r ,计算的方式由公式(2-11)可知,是沿视点 to 视线终点的路径,对大气密度的线积分。对于内散射辐射率 L_i ,由于要考虑间接光的内辐射,计算比较复杂,但依旧只与视点的位置,视线的方向,大气的密度分布,以及太阳光的方向有关。如果从宏观的层面考虑,忽略大气密度在水平方向上相对较小的不均匀性,只考虑大气层的密度关于海拔高度变化,就可以对这两项的所有可能值进行预先的计算并保存计算结果,最后在实时渲染的阶段根据场景和视点的变化,直接查找到对应的结果,提升渲染的效率。由于大气散射的最终效果,是一个低频的全局光照,因此不需要大量存储空间记录非常多的计算结果,算法具有在空间和时间上的可行性。

尽管算法在时间上的复杂度并不算太高,但仍旧涉及了多个积分的计算,为了提升计算的效率,同时顾及实时渲染阶段对数据的访问,算法将会利用图形处理器(Graphics Processing Unit, GPU)的并行计算能力,使用计算着色器,在 GPU 上进行计算。得到的结果将以二维或三维纹理的形式,保存到 GPU 的显存中,方便 GPU 渲染时访问

3.2 透射率的预计算算法

本节将介绍透射率计算的算法。该算法可以计算出大气层内任意高度的视点到大气层边界各处的透射率,并将结果存储在纹理内,以便计算其他预计算变量以及实时渲染时使用。

由公式(2-11)可知,透射率 $T_r(x' \leftrightarrow x)$ 具有如公式(3-1)所示的性质,即对于空间中直线上的三点 x, x', x'' ,点 x 和点 x'' 之间的透射率 $T_r(x \leftrightarrow x'')$ 为其路径中任意一点的到两端的透射率之积。具体示意图如图 3-1 所示。

$$T_r(x \leftrightarrow x'') = T_r(x \leftrightarrow x')T_r(x' \leftrightarrow x'') \quad (3-1)$$

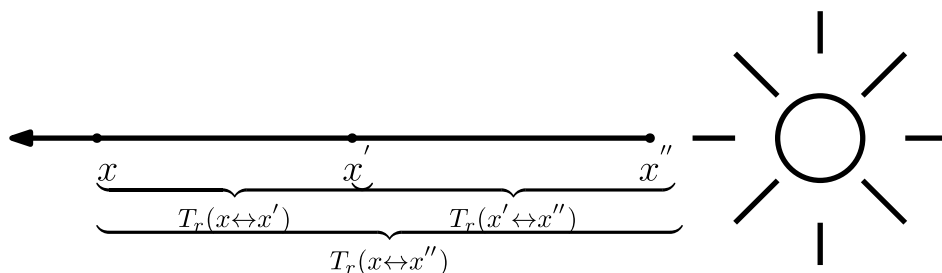


图 3-1 透射率性质示意图

根据透射率的该性质,并假设大气层内气体在相同海拔上均匀分布,就可以根据球体的对称性,将三维空间内的透射率计算简化到二维的平面内,并在预处理阶段计算出各海拔到大气层上边界各处的透射率,记录计算结果,在实时渲染阶段通过对结果的查找,快速计算出任意两点间的透射率。

3.2.1 消光系数的确定

由 2.2.1 节可知,透射的产生主要由消光现象引起。大气消光有三种主要的成分:气体分子的瑞利散射(Rayleigh Scattering)、悬浮微粒的米氏散射(Mie Scattering)和分子的吸收^[27]。其中,瑞利散射由瑞利勋爵于 1871 年提出,描述的是半径小于十分之一波长的微粒对光的散射^[28]。瑞利散射的散射强度与光的波长以及分子密度相关,如公式(3-2)所示^[6]:

$$\sigma_s^R(h, \lambda) = \frac{8\pi^3(n^2 - 1)^2}{3N\lambda^4} e^{-\frac{h}{H_R}} \quad (3-2)$$

其中, λ 为波长, h 为海拔, n 为空气的折射率, N 为海平面处的分子密度, H_R 为匀质大气层的厚度。

瑞利散射带来的结果是,波长较短的蓝光被散射得更多,在晴朗的白天,天空呈现蓝色;而在清晨和黄昏,由于太阳角度的变化,光线在到达地面时经过了更长的路径,太阳光中波长较短的部分的能量在到达地面前就已经被散射完,因此天空呈现波长较长的红色。

对于直径稍大一些的微粒,例如空气中的水珠,灰尘等,则主要发生米氏散射。米氏散射的强度于波长无关,只与微粒的密度相关,且散射的角度分布也大致相同^[29]。米氏散射的散射系数如公式(3-3)所示^[6]:

$$\sigma_s^M(h) = \sigma_s^M(0) e^{-\frac{h}{H_M}} \quad (3-3)$$

其中, h 为海拔, H_M 为含有发生米氏散射的微粒的大气层厚度。

大气层中,臭氧对太阳光中的一些波段具有强烈的吸收作用,因此在确定消光系数

的时候也需要考虑臭氧的影响。臭氧分子密度随海拔高度的变化如图 3-2 所示。

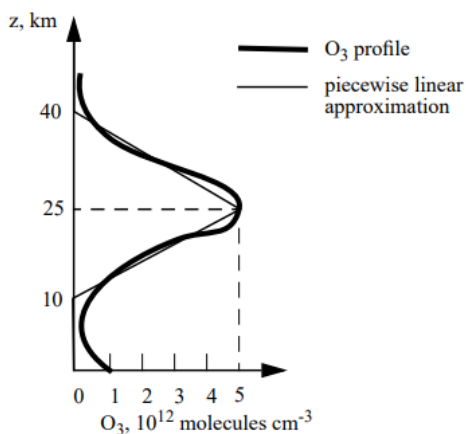


图 3-2 臭氧分子密度随海拔高度的变化^[30]

由图 3-2 可知, 可以用分段直线近似地描述臭氧分子密度的变化。对于臭氧的消光系数, 根据比尔-朗伯定律, 可以用公式 (3-4) 计算^[31]:

$$\sigma_a^{ozone}(h, \lambda) = N_{ozone}(h) \times crossSection(\lambda) \quad (3-4)$$

其中, $N_{ozone}(h)$ 为臭氧分子的数量, $crossSection(\lambda)$ 为臭氧的吸收横截面积。给出臭氧分子数量随海拔的分布以及吸收横截面积关于光的波长的变化, 就可以计算出对应高度的臭氧消光系数。

由上可以计算出大气的消光系数 $\sigma_a(h, \lambda)$:

$$\begin{cases} \sigma_a(h, \lambda) = \sigma_a^R(h, \lambda) + \sigma_a^M(h) + \sigma_a^{ozone}(h, \lambda) \\ \sigma_a^R = \sigma_s^R \\ \sigma_a^M = 1.1\sigma_s^M \end{cases} \quad (3-5)$$

3.2.2 透射率的计算

在进行计算的时候, 可以假定大气层具有对称性, 即相同海拔的大气是均匀的, 这样一来就可以忽略视点在地球表面的位移, 只考虑视点海拔高度, 在地球的经面上进行计算。如图 3-3 所示。

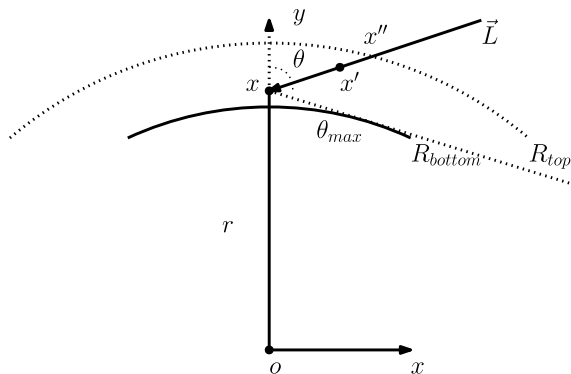


图 3-3 透射率计算示意图

其中, $\vec{L}$ 为光线的传播方向, x 为大气层内一点, o 为地心, r 为 ox 的长度, R_{top} 和 R_{bottom} 分别为大气层的上下边界, θ 为 $\vec{L}$ 和的 ox 夹角。由 2.2.1 节与 3.1.1 节可知, 对点 x 沿 $\vec{L}$ 方向到与大气层边界的交点 x'' 之间的透射系数 $T_r(x \leftrightarrow x'')$ 可由公式 (3-6) 计算得到。其中 $h(t)$ 为海拔高度关于积分路径的函数。

$$T_r(x \leftrightarrow x'') = \exp \left\{ - \int_x^{x''=x-t\vec{L}} [\sigma_a^R(h(t), \lambda) + \sigma_a^M(h(t)) + \sigma_a^{ozone}(h(t), \lambda)] dt \right\} \quad (3-6)$$

由于公式 (3-6) 中指数部分的线积分难以求得解析解, 因此可以使用梯形法则, 对于给定的传播方向 $\vec{L}$ 和大气层内一点 x , 对点 x 沿 $\vec{L}$ 方向到与大气层边界的交点 x'' , 按照固定的步长, 计算出路径上各点的消光系数 σ_a , 并乘以步长, 最后计算出线积分的数值解。具体的计算方法如公式 (3-7) 所示。其中, 对于不同的波长, 可以选取红绿蓝三色的波长进行计算, 也可以对可见光波段的光进行分别计算。

$$T_r(x \leftrightarrow x'') \approx \exp \left\{ - \frac{|x''x|}{N} \left[\frac{\sigma_a(x) + \sigma_a(x'')}{2} + \sum_{i=1}^{N-1} \sigma_a \left(h \left(x + i \frac{|x''-x|}{N} \right), \lambda \right) \right] \right\} \quad (3-7)$$

$$\sigma_a(x) = \sigma_a^R(x) + \sigma_a^M(x) + \sigma_a^{ozone}(x)$$

在预计算阶段, 对于 $r \in [R_{bottom}, R_{top}]$ 的大气层内任意一点 x , 都可以对其所有夹角 $\theta \in [0, \theta_{max}]$ 的光线按照公式 (3-7) 计算其到大气层上边界的透射率。其中 θ_{max} 为光线恰好沿地平线射入时的夹角。

计算出透射率后, 需要将其结果进行存储。可以看到, 视点的位置可以使用变量 r 确定, 光线的方向则和角度 θ 有关, 因此透射率是关于视点高度 r 和光线角度 θ 两个变量的函数, 因此可以将计算结果存储到一张二维的纹理上。

3.3 内散射的预计算算法

内散射的计算较为复杂, 主要可以分为直接散射和间接散射两部分, 直接散射是指太阳光经过散射后, 直接入射视点的辐射率, 类似于表面渲染模型中的直接光照; 间接散射则计算的是太阳光经过散射后, 那些没有入射视点的散射光在经过多次散射后, 又重新入射视点的辐射率, 类似于表面渲染模型中的间接光照。间接散射的计算可以分解为多级散射, 每一级的计算都依赖于上一级的计算结果。本节将介绍这两种散射的计算算法。

3.3.1 直接散射的预计算算法

本节将介绍直接散射的计算算法, 该算法可以对所有的可能的太阳光入射角度和大

气层内任意高度的视点，都计算出视点到大气层边界各处的直接散射辐射率之和，并将结果存储在纹理中，以便于实时渲染时，通过这三个变量，直接查找出所需的结果。

图 3-4 为当太阳光以 $\vec{L}_{sun}$ 方向入射时，由路径 $\overline{x'x}$ 经直接散射进入视点 x 的辐射率示意图。即直接散射只考虑太阳光经过散射改变方向后，直接进入视点的辐射率。

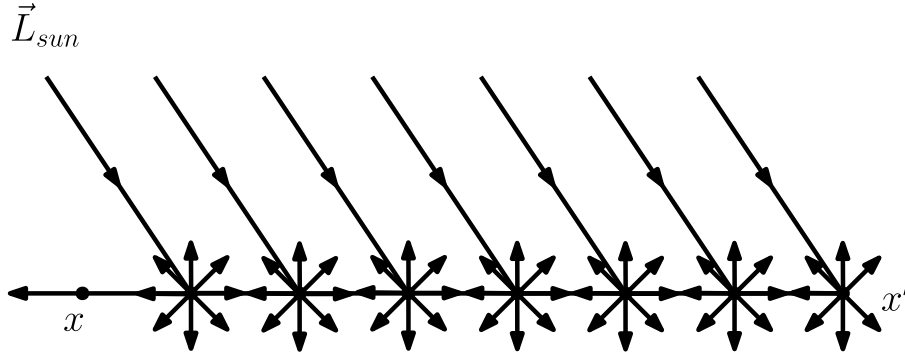


图 3-4 直接散射示意图

与透射率只表示光线在两点之间的能量比例不同，散射的计算与太阳的天顶角以及太阳光的入射能量有关。直接散射的计算示意图如图 3-5 所示。其中点 x 为视点， $\vec{v}$ 为视线的方向， x'' 为视线与大气层上边界的交点， x' 为是视线上的点， $\vec{L}_{sun}$ 为太阳光的方向， s' 为 x' 沿 $\vec{L}_{sun}$ 方向与大气上边界的交点， o 为地心， α 为太阳光与 ox 的夹角， θ 为视线与 ox 的夹角。

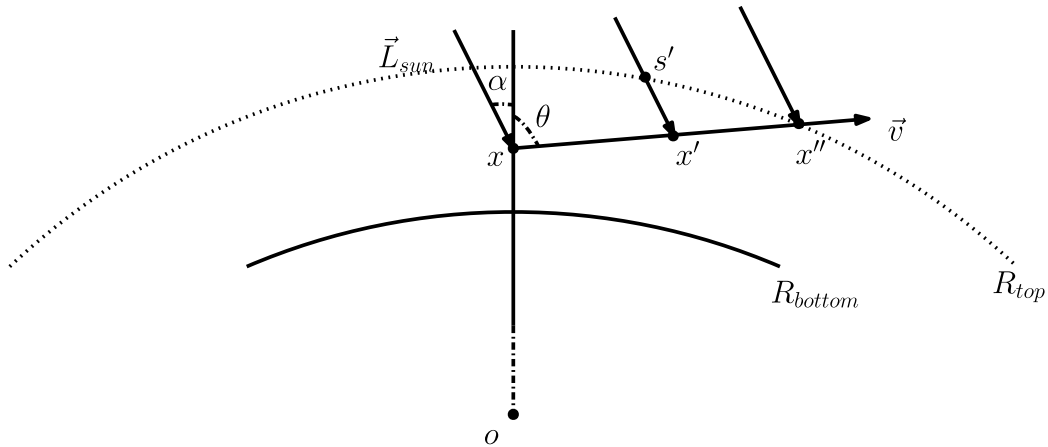


图 3-5 直接散射的计算示意图

对于视点 x ，视点受到的在视线 $\vec{v}$ 上一点 x' 处发生散射并直接射入的太阳光的辐射率 $L(x' \rightarrow x)$ ，可以用公式 (3-8) 计算得到：

$$L(x' \rightarrow x) = T_r(x \leftrightarrow x') L_{sun} T_r(x' \leftrightarrow s') [\sigma_s^R p_R(\alpha + \theta) + \sigma_s^M p_M(g, \alpha + \theta)] \quad (3-8)$$

其中， L_{sun} 为太阳光在大气层顶部的辐射率，由于可以假设外太空为真空，不存在能量的散射和吸收，因此 L_{sun} 在大气层顶部的各处均相等。 $p_R(\alpha + \theta)$ 和 $p_M(g, \alpha + \theta)$ 分

别为瑞利散射和米氏散射的相函数，其中瑞利散射的相函数如公式（3-9）所示^[6]：

$$p_R(\theta) = \frac{3}{16\pi}(1 + \cos^2 \theta) \quad (3-9)$$

米氏散射的相函数如公式（3-10）所示^[6]：

$$p_M(g, \theta) = \frac{3(1 - g^2)(1 + \cos^2 \theta)}{8\pi(2 + g^2)(1 + g^2 - 2g \cos \theta)^{1.5}} \quad (3-10)$$

求视线 $\vec{v}$ 上各点太阳光经直接散射进入视点 x 的辐射率之和，就可以通过对 $L(x' \rightarrow x)$ 进行线积分得到，如公式（3-11）所示：

$$L(x'' \rightarrow x) = \int_x^{x''=x+t\vec{v}} L(x'(t) \rightarrow x) dt \quad (3-11)$$

和计算透射率的方法相同，可以使用梯形法则求解积分的数值解。求解时，由于散射的相函数的结果为常数（角度 $\alpha + \theta$ 在积分过程中保持不变），且相函数的角精度对最终的渲染结果影响较大，因此可以移除相函数项，在实时渲染的阶段再进行计算。具体的数值解计算方法如公式（3-12）所示：

$$L(x \rightarrow x'') \approx \frac{|x''x|}{N} \left[\frac{L(x \rightarrow x) + L(x'' \rightarrow x)}{2} + \sum_{i=1}^{N-1} L(x'(i) \rightarrow x) \right] \quad (3-12)$$

由于相较于透射率的计算，直接散射的计算还引入了太阳角度 α ，因此，直接散射是关于视点高度 ox ，视线与 ox 夹角 θ ，以及太阳与 ox 夹角 α 的函数。可以三个自变量归一化后，将结果存入三维纹理。

3.3.2 间接散射的预计算算法

由于被散射的阳光会向空间中多个方向投射，这些光在遇到微粒之后又会继续向四周散射，并有可能重新进入视点，因此如果只计算直接散射，会使得最终的渲染结果较暗。图 3-6 为间接散射的示意图。图中的点 $x_0, x_1, x_2, \dots$ 为路径 $\overrightarrow{xx'}$ 上的点，围绕点的虚线圆和实线箭头表示该点受到了来自点附近球面上各个点散射来的光照，这些光又分别向点 x 散射，称这一级散射为 n 级散射 S_n 。引发 n 级散射的每个点又收到附近球面上各个点的 S_{n-1} 级散射……，依此类推，直到产生第 S_1 级散射的各个点受到了由太阳光直接散射来光照。因此 3.2 节中介绍的直接散射也可以被记作 S_0 级散射。

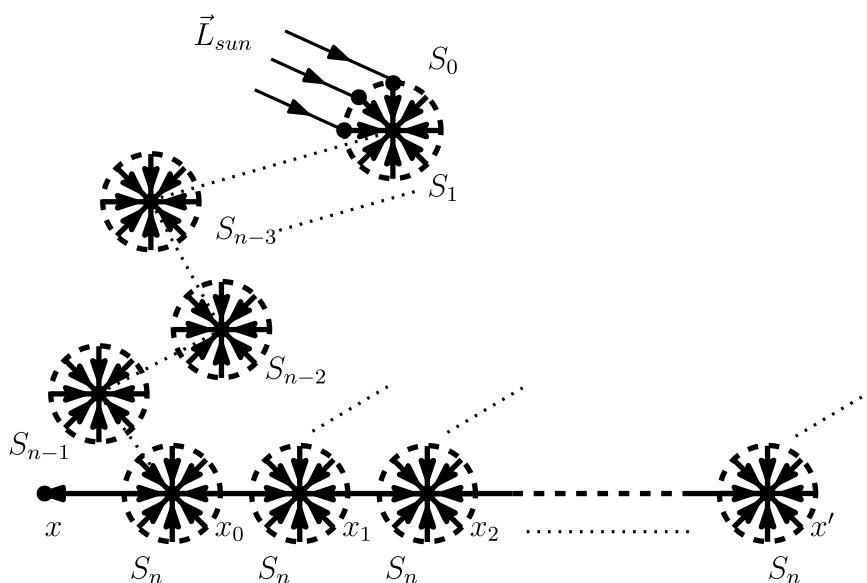


图 3-6 间接散射示意图

由此可知，计算视点 x 受到的来自视线方向 $\vec{\omega}$ 上点 x_0 的第 S_n 级散射的间接辐射率 $L_{S_n}(x_0 \rightarrow x)$ 的公式可以由 (3-13) 表示：

$$L_{S_n}(x_0 \rightarrow x) = \int_{\Omega=4\pi} L_{S_{n-1}}(x_0 \leftarrow \vec{\omega}') \left(\sigma_s^R(h_{x_0}) p_R(\theta') + \sigma_s^M(h_{x_0}) p_M(\theta') \right) d\vec{\omega}' \quad (3-13)$$

其中， $L_{S_{n-1}}(x_0 \leftarrow \vec{\omega}')$ 表示点 x_0 受到的来自周围 $\vec{\omega}'$ 方向的第 S_{n-1} 级散射的辐射率，积分式的另一部分则计算了来自 $\vec{\omega}'$ 方向的第 S_{n-1} 级散射光往 $\vec{\omega}$ 方向出射的辐射率比例。最后，沿 x_0 附近的球面计算出射辐射率的球面积分，得到最终的计算结果。该等式是一个递推等式，特别地， L_{S_0} 的计算方法已由节 3.3.1 公式 (3-11) 给出。

完成 $L_{S_n}(x_0 \rightarrow x)$ 的计算后，还需要沿线方向 $\vec{\omega}$ 对各点进行线积分，才能得到大气层边界到视点 x 之间，第 S_n 级散射的间接辐射率的总和。具体计算公式如 (3-14) 所示：

$$L_{S_n}(x' \rightarrow x) = \int_x^{x'=x+\vec{\omega}} L_{S_{n-1}}(x'(t) \rightarrow x) T_r(x'(t) \leftrightarrow x) dt \quad (3-14)$$

在求解方程的数值解时，由于计算第 S_n 级散射时，需要计算第 S_{n-1} 级散射的值，因此需要在完成第 S_{n-1} 级散射的计算后，将计算结果保存至纹理中。而在计算的过程中，如图 3-7 所示，如果直接按照公式 (3-14) 计算出视点 x 处的 $L_{S_n}(x_N \rightarrow x)$ ，那么在计算的过程中会计算出路径上点 x_i, x_{i+1} 处的第 S_n 级散射辐射率 $L_{S_n}(x_N \rightarrow x_i), L_{S_n}(x_N \rightarrow x_{i+1})$ 。而当视点移动到 x_i 处时，又会重复计算 $L_{S_n}(x_N \rightarrow x_{i+1})$ 。因此，为了避免重复计算，应该按照公式 (3-13)，对路径上的每个点，计算该点向视点散射的第 S_n 级散射辐射率，并记录计算的结果。完成各点的计算后在根据保存的计算结果，计算视点到大气层边界的线积分，得到最终的计算结果，并进行记录，以便进行实时渲染时进行查询。

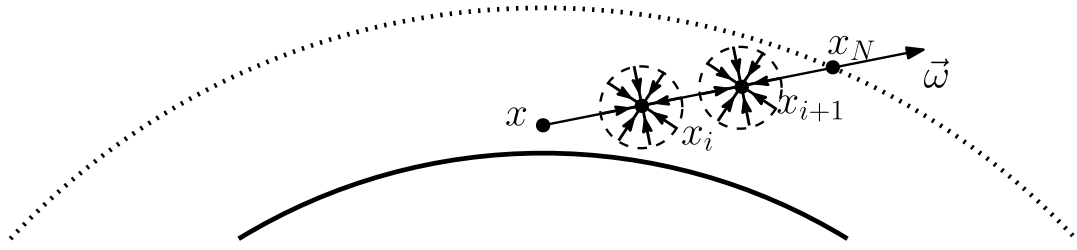


图 3-7 间接散射计算示意图

计算数值解时，可以将公式 (3-13) 的球面积分可以转化为如下所示：

$$L_{S_n}(x_k \rightarrow x) \approx \sum_{i=0}^N \sum_{j=0}^{2N} L_{S_{n-1}}(x_k \leftarrow \vec{\omega}_{ij}) \left(\sigma_s^R p_R(\theta_{ij}) + \sigma_s^M p_M(\theta_{ij}) \right) \frac{\pi^2}{N^2} \sin \frac{i\pi}{N} \quad (3-15)$$

其中， $\vec{\omega}_{ij}$ 是球面上一点指向点 x_k 的向量，计算如公式 (3-16) 所示：

$$\vec{\omega}_{ij} = \left(\cos \frac{j\pi}{N} \sin \frac{i\pi}{N}, \sin \frac{j\pi}{N} \sin \frac{i\pi}{N}, \cos \frac{i\pi}{N} \right) \quad (3-16)$$

θ_{ij} 为 $\vec{\omega}_{ij}$ 与 $\overrightarrow{x_k x}$ 之间的夹角，可由公式 (3-17) 计算得到：

$$\cos \theta_{ij} = \text{dot}(\overrightarrow{x_k x}, \vec{\omega}_{ij}) \quad (3-17)$$

公式 (3-14) 可以转化为公式 (3-18) 计算其数值解：

$$L_{S_n}(x_N \rightarrow x) \approx \frac{|x x_N|}{N} \sum_{k=1}^{N-1} L_{S_{n-1}}(x_k \rightarrow x) T(x_k \leftrightarrow x) \quad (3-18)$$

由公式 (3-13) 和公式 (3-14) 可知，间接散射的值与视点的高度，视线的方向 $\vec{\omega}$ 以及太阳光的方向有关。因此可以将三个自变量归一化后，将结果存入三维纹理中。

3.4 本章小结

本章主要对天空渲染的体渲染方程的计算方法进行了介绍，将体渲染方程的各项进行了分析，在大气分布大致均匀以及对称的基础上，对体渲染方程中的透射率以及内散射项在非实时处理阶段进行了计算，对每个视点可能的高度，每个视线可能的方向，以及太阳的天顶角，进行了离散化，并计算出相应的查找表，以供实时渲染阶段根据实际的相机位置和太阳方向，通过查找和插值确定对应的结果。并设计了对应的场景渲染算法，针对不同的渲染管线，对物体和天空进行着色。

第4章 体积云的渲染技术

4.1 噪声纹理在体积云渲染中的应用

云朵是由高空中的水蒸气在上升过程中遇到冷空气,凝结成冰晶或水滴悬浮空气中形成。云朵的形状会随着气流的变化而快速变化,同时因为微粒的聚集,会在表面呈现出复杂的细节,如图 4-1 所示。由于这些原因,用表面模型去描述云朵的难度非常大。



图 4-1 云朵形状照片^[32]

目前主流的方案是使用体渲染对云进行渲染,利用 2-3 节所提出的体渲染方程对云的颜色进行计算。对于云层中微粒的密度,则使用随机生成的程序式纹理,即噪声纹理进行描述。本文中主要使用的噪声纹理为 Perlin 噪声和 Worley 噪声。

4.1.1 Perlin 噪声

传统的随机噪声为每个像素计算一个随机的灰度值,这种噪声为白噪声。白噪声与自然界物体的随机性之间有着很大的差异。Ken Perlin 于 1985 年对白噪声进行了改进,指出若要对自然界的随机外观进行模拟,噪声应该具有对旋转具有统计不变性;能量在频谱上集中于一个窄带,即图像是连续的,高频分量受限;对变换具有统计不变性的性质^[33]。Ken Perlin 按照这些性质,设计了生成 Perlin 噪声的算法。白噪声和 Perlin 噪声以及他们的频谱如图 4-2 所示。Perlin 噪声可以用来模拟自然界中的噪声现象,由于它的连续性,如果以二维噪声中的一个轴作为时间轴,可以得到一个连续变化的一维函数。同样的也可以得到连续变化的二维图像。该噪声可以用来模拟水的波纹,岩石的

形状，山峦的排布等自然现象。

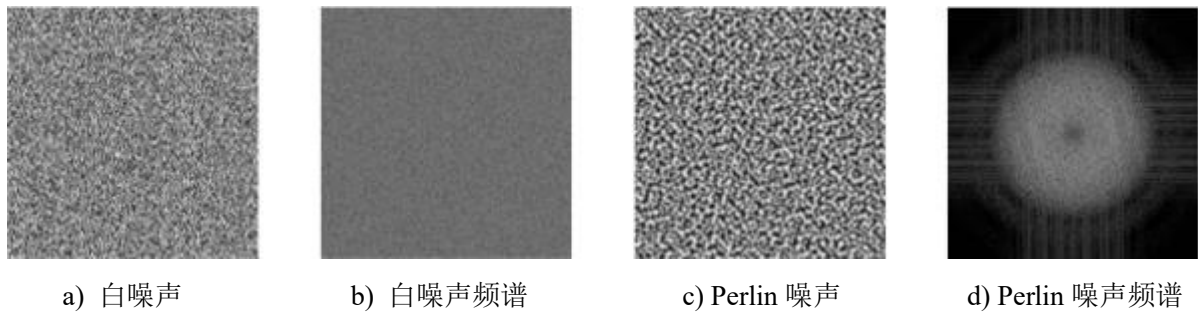


图 4-2 白噪声与 Perlin 噪声及其各自频谱^[33]

由于 Perlin 噪声各个值的计算相互之间是独立的，因此非常适合使用图形处理器进行并行计算。Khronos 委员会在 GLSL 标准中为最多四个维度的 Perlin 噪声提供了内建支持，这些噪声的性质与上述 Perlin 所提出的性质相同。不过在 HLSL Shader Model 2.0 以上的标准中，并没有提供相应的 Perlin 噪声接口，因此本文需要使用 GPU 自行生成噪声。

Perlin 噪声的生成非常简单，也可以很容易地拓展到高维，三维 Perlin 噪声的生成示意图如图 4-3 所示：

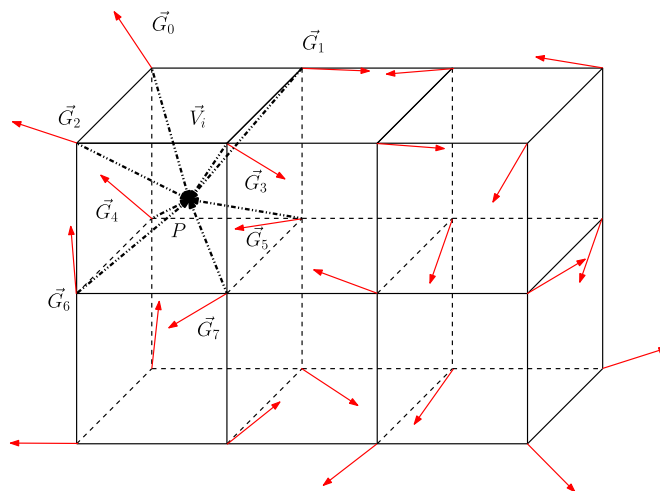


图 4-3 Perlin 噪声的生成示意图

计算三维 Perlin 噪声时，首先将空间划分为边长为 1 的立方体网格，对于网格的每个顶点，生成一个随机的三维梯度向量 $\vec{G}_i$ ，对于网格内要计算噪声值的一点 P ，将对应立方体各个顶点的随机梯度 $\vec{G}_i$ 投影到 P 相对于立方体顶点的向量 $\vec{V}_i$ ，得到该梯度对噪声值的贡献 N_i ，如公式（4-1）所示：

$$N_i = \text{dot}(\vec{G}_i, \vec{V}_i) \quad (4-1)$$

之后需要对 N_i 在三个维度上进行插值，得到最终的噪声值 N_p ，具体公式如下^[34]：

$$\begin{cases} N_{x_i} = \text{lerp}_p(N_{2i}, N_{2i+1}, P.x - \lfloor P.x \rfloor), i = 0, 1, 2, 3 \\ N_{y_i} = \text{lerp}_p(N_{x_{2i}}, N_{x_{2i+1}}, P.y - \lfloor P.y \rfloor), i = 0, 1 \\ N_p = \text{lerp}_p(N_{y_0}, N_{y_1}, P.z - \lfloor P.z \rfloor) \end{cases} \quad (4-2)$$

其中 $\text{lerp}_p(a, b, x)$ 为 Perlin 噪声所使用的插值函数，如公式 (4-3) 所示^[34]：

$$\text{lerp}_p(a, b, x) = a(1 - 6x^5 + 15x^4 - 10x^3) + b(6x^5 - 15x^4 + 10x^3) \quad (4-3)$$

生成的三维 Perlin 噪声的一个切面如图 4-3 所示。

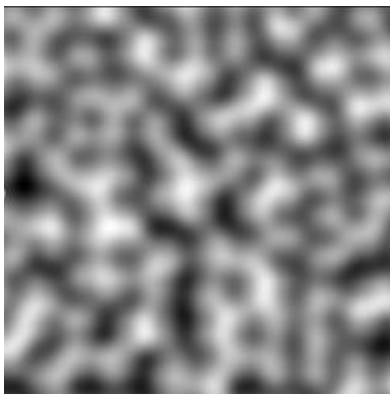


图 4-4 生成的 Perlin 噪声

4.1.2 Worley 噪声

Worley 噪声由 Steven Worley 于 1996 年发明，是一种基于空间分割算法生成的噪声，主要用于对石板表面，岩石，焦散等现象的模拟。

Worley 噪声的计算也非常适合使用图形处理器进行并行计算，生成三维 Worley 噪声的示意图如图 4-5 所示：

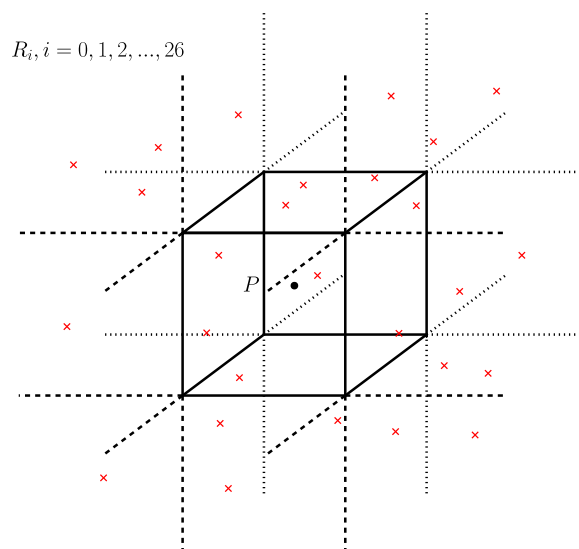


图 4-5 Worley 噪声生成示意图

计算三维 Worley 噪声时，首先将空间划分为边长为 1 的立方体网格，对于每个网

格内的点 P 以及周围 3×3 的网格,都在网格内部生成一个随机的特征点 R_i ,对每个点 R_i ,计算其到点 P 的距离,并取最小值为 Worley 噪声的输出值,具体计算如公式(4-4)所示^[35]。

$$N_W = \min\{i \in [0,26], i \in \mathbb{N} \mid \|\overrightarrow{R_i P}\|\} \quad (4-4)$$

对由公式(4-4)计算得到的 Worley 噪声进行反色处理,可以得到类似于分子构型的图案。生成的三维反色 Worley 噪声的一个切面如图 4-6 所示。

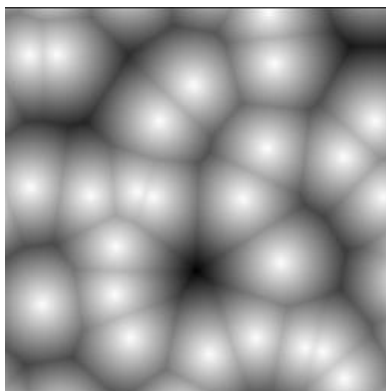


图 4-6 生成的反色 Worley 噪声

如果从特征点而不是像素点的角度对算法进行解释,则可以理解为:每个特征点向外扩张生长,直到它碰到其它特征点所控制的区域。这反映了自然界的生长规则,生命的形态是由内部扩张、生长的力量和限制性的外部力量共同决定的。模拟这种行为的算法也叫做 Voronoi 算法^[36]。

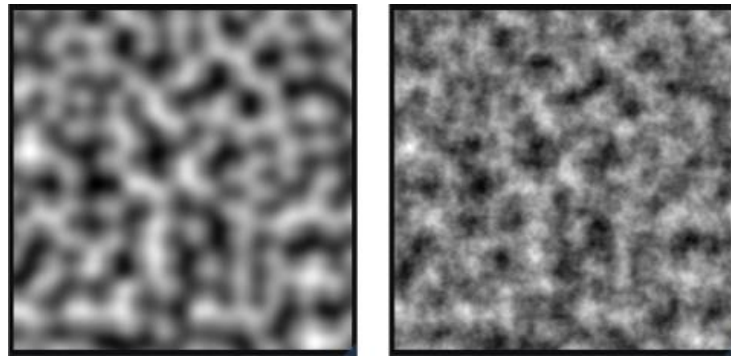
4.1.3 分形噪声

分形通常被定义为“一个粗糙或零碎的几何形状,可以分成多个部分,且每一部分都(至少近似地)是整体缩小后的形状”,即具有自相似的性质。分形在数学中是一种抽象的物体,用于描述自然界中存在的事物^[37],例如云层,海岸线等。

在噪声纹理中应用分形非常简单,只需要将不同频率的噪声加权之后相加即可,具体公式如(4-5)所示:

$$fbm(noise(p), O, G, L) = \sum_{i=0}^{O-1} noise(p \times L^i) \times G^i \quad (4-5)$$

对 Perlin 噪声进行分形处理,得到的结果如图 4-7b 所示:

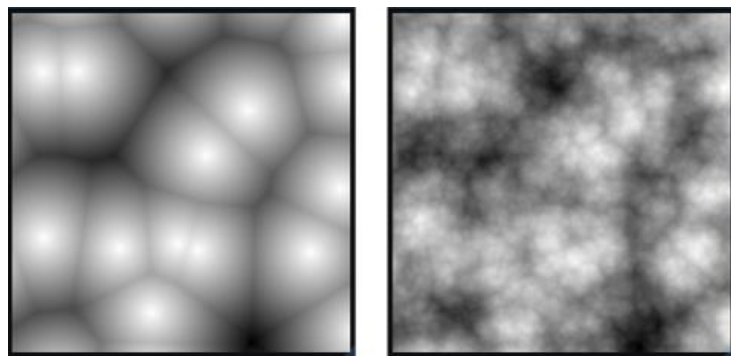


a) Perlin 噪声

b) 分形 Perlin 噪声

图 4-7 Perlin 噪声与其对应的分形噪声

对反色 Worley 噪声进行分形处理，得到的结果如图 4-8b 所示：



a) 反色 Worley 噪声

b) 反色分形 Worley 噪声

图 4-8 反色 Worley 噪声及其分形噪声

通过增加公式 (4-5) 中的参数 O 的值，可以增加图形的细节，图 4-9abc 分别展示了 $O = 2, 3, 4$ 时的反色分形 Worley 噪声。

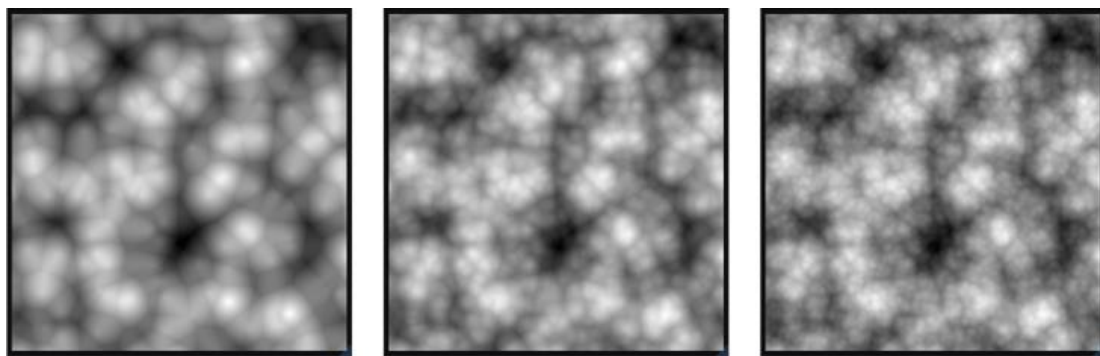
a) $O = 2$ 时的分形噪声b) $O = 3$ 时的分形噪声c) $O = 4$ 时的分形噪声

图 4-9 不同参数的反色 Worley 分形噪声

由于云的形状可以由 Perlin 以及 Worley 的分形噪声一同确定，Perlin 分形噪声可以用于确定云的雾状形状，但对于云层中翻滚，类似于菜花的形状则难以表现。Worley 的分形噪声则带有这种翻滚的波浪的形状，因此可以用于为云层增加细节。将两种噪声按公式 (4-6) 混合，得到 Perlin-Worley 噪声。该式在 Andrew Schneider 提出的混合公

式上改进而来^[14]，避免在噪声中出现频率的激变。

$$N_{P-W} = \max\left(0, \min\left(1, \frac{P_{fbm} - W_{fbm}}{1 - W_{fbm}}\right)\right) \quad (4-6)$$

生成的 Perlin-Worley 噪声如图 4-10 所示：

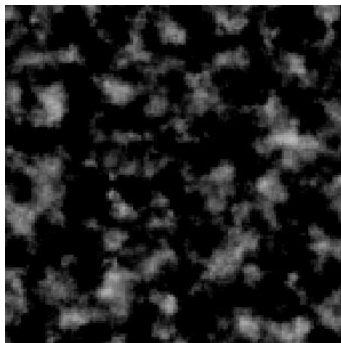


图 4-10 Perlin-Worley 噪声

4.2 云层的生成

在生成了噪声纹理之后，就可以使用噪声纹理生成云层的基本形状。云层的形状可以用低频率的 Worley 分形噪声叠加与 Perlin-Worley 噪声混合而形成，具体的计算方法如公式（4-7）所示^[38]：

$$base(p) = \frac{\max\left(0, \min\left(1, \left(N_{P-W}(p) - W_{fbm}(p)\right)\right)\right)}{1 - W_{fbm}(p)} \quad (4-7)$$

在确定云层的基本形状之后可以使用高频的 Worley 分形噪声增加一些细节，计算方法与公式（4-7）类似：

$$shape(p) = \frac{\max\left(0, \min\left(1, \left(base(p) - W_{fbm}(p)\right)\right)\right)}{1 - W_{fbm}(p)} \quad (4-8)$$

在确定了云的基本形状后，还可以使用噪声生成云层的分布图，云层的分布需要分别从水平分布和垂直分布两个维度进行考虑，对于云层的水平分布，可以使用范围在 [0,1] 的覆盖率标记改点的云层密度。可以使用 Perlin-Worley 噪声生成一张二维纹理，充当云层的水平分布图，如图 4-11 所示。

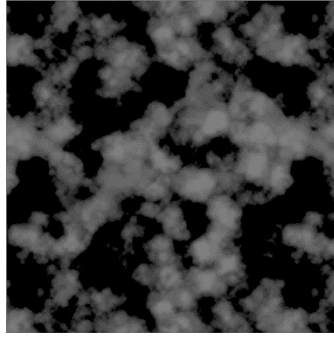


图 4-11 云层水平分布图

云层的水平分布可以使用公式 (4-9) 进行计算:

$$clouds(p) = coverage_h(p.xz) \times shape(p) \quad (4-9)$$

本文使用体渲染技术渲染三种类型的云, 层云(Stratus), 积云(Cumulus)和积雨云(Cumulonimbus), 这些种类的云在垂直方向上的分布大致如图 4-12 所示。其中层云较薄, 大约分布在 1.5 千米海拔左右的空中; 积云大致分布在 1.5~4 千米海拔左右的空中; 积雨云则可以分布在 1~8km 的空中。

a) 层云^[39]b) 积云^[40]c) 积雨云^[41]

图 4-12 三种常见的不同类型云层的照片

对这三类云的描述可以简单地使用公式 (4-10) 进行描述:

$$type_i(p.y) = \begin{cases} 0, p.y < c_{bot} \\ smoothstep(c_{bot}, s_{bot}, p.y), c_{bot} < y < s_{bot} \\ 1, s_{bot} < y < s_{top} \\ smoothstep(s_{top}, c_{top}, p.y), s_{top} < y < c_{top} \\ 0, y > c_{top} \end{cases} \quad (4-10)$$

其中, $smoothstep(a, b, x)$ 为 HLSL 着色语言内建的插值函数。 c_{bot} 和 c_{top} 分别为该种类的云出现的海拔最大以及最小值。 s_{bot} 和 s_{top} 为云层密度不发生变化的范围。通过对 c_{bot} , c_{top} , s_{bot} , s_{top} 四个参数进行调整, 就可以模拟不同类型的云层, 密度随海拔高度的变化。

有了云层的垂直密度函数后, 就可以定义云的类型参数, 其中层云的值为 0, 层积云的值为 0.5, 积云的值为 1。使用低频的 Perlin-Worley 噪声生成云层类型的分布图, 如图 4-13 所示:

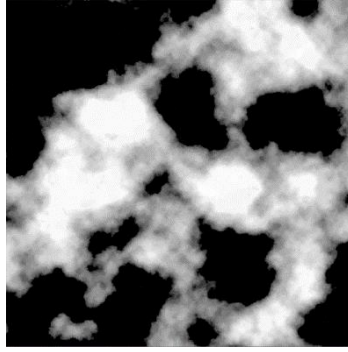


图 4-13 云层类型分布图

对于介于(0,0.5)和(0.5,1)的云层类型值 t ，可以使用相邻云层的计算公式插值得到，具体计算如公式（4-11）所示：

$$coverage_v(p.y,t) = lerp\left(type_i(p.y), type_{i+1}(p.y), \frac{t - T_i}{0.5}\right) \quad (4-11)$$

其中 $T_i = 0, 0.5, 1$ ，取小于 t 的最大值。

自然界中，云层还会因为空气的对流而运动。空气的对流随着海拔的升高而增强，表现的形式则为在高海拔处的云移动比低海拔处的云移动快。为了实现云层运动的效果，可以引入风向和风速，使用这两个量对云的位置进行偏移；同时，利用海拔高度对风速的大小进行调整，来达到不同高度，云的运动速度不同的效果。具体的计算方式如公式（4-12）所示。其中 p_{origin} 为采样点处云层的原始位置， $\vec{w}$ 为风向的向量， s_w 为风速的大小。

$$p = p_{origin} + \vec{w} \times s_w \times \frac{p_{origin.y}}{R_{top}} \quad (4-12)$$

4.3 体积云的光照模型

云层的渲染方程与第二章中的公式（2-16）给出的体渲染方程相同，但与大气层不同的是，云层的密度分布不均匀，且变化较快，无法用低分辨率的预渲染预计算积分项，只能通过一些手段在实时渲染计算出近似值或者估计值，本节将要介绍体积云的光照计算方法。

来自物体表面反射的辐射率 L 可以通过对生成的天空颜色缓存进行纹理采样得到。透射率 T_r 则需要使用一种被称为 Ray-March 的技术，具体的方式为：在光路方向上进行步进，并在每次步进对步进点附近的云层密度进行采样，并由此计算其光学深度 τ_r 。具体的计算方法如公式（4-13）所示：

$$\tau_r(x \leftrightarrow x_s) = - \sum_{i=0}^N density\left(x + i \frac{\overrightarrow{xx_s}}{N}\right) \times \frac{|\overrightarrow{xx_s}|}{N} \quad (4-13)$$

透射率 T_r 的计算如公式(4-14)所示:

$$T_r(x \leftrightarrow x_s) = e^{-\tau(x \leftrightarrow x_s)} \quad (4-14)$$

内散射辐射率 L_i 依旧分为直接散射和间接散射进行分别计算,对于由直接散射进入视点的辐射率,需要对太阳光在采样点散射后进入视点比例,以及太阳光在传播路径中的能量损失进行计算。太阳光 $\vec{S}$ 从云层外到达采样点 x_k 的辐射率可以用公式(4-15)计算:

$$L(x_k) = L_{sun} e^{-\tau_s(x_k)} \quad (4-15)$$

其中 τ_s 为采样点沿 $-\vec{S}$ 方向的光学深度,为减少渲染的负担,只在光源方向上做7次采样,计算方式如公式(4-16)所示:

$$\tau_s(x_k) = \sum_{i=0}^6 \text{density}(x_k + \vec{v}(i)) \times \text{stepSize} \quad (4-16)$$

为了估计在采样点附近发生的内散射的影响, $\vec{v}(i)$ 并不是沿光源方向直线前进的函数,而是在朝向光源方向的圆锥内随机分布的函数^[14]。如图4-14所示。

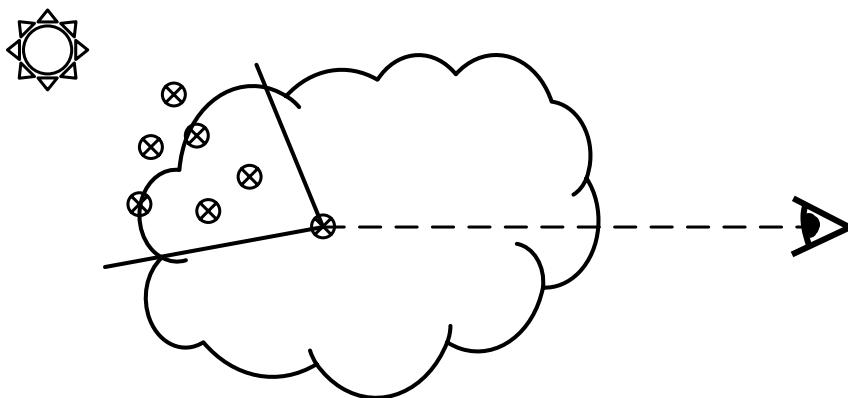


图4-14 光源方向采样示意图^[14]

太阳光在采样点发生散射后,进入视点的辐射率计算方法如公式(4-17)所示:

$$L(x_k \rightarrow x) = L(x_k) p_{HG}(g, \theta) T_r(x \leftrightarrow x_k) \quad (4-17)$$

对于由间接散射进入视点的辐射率,由于直接求解渲染方程的数值解的运算量太大,不能在实时渲染阶段进行计算,因此可以从间接光经过散射后进入视点的概率的角度进行考虑。

内散射只会发生在云的液滴悬浮的地方,因此在云层较厚的地方,会有较多的间接散射发生,在云层较薄的地方则相反。同时,来自云层的间接散射都是太阳光在其内部多次散射的结果,因此在云层的底部不受阳光直射的一侧,间接散射的强度会比在云层顶部小。确定了云层厚度,高度与间接散射强度的大致关系后,就可以使用这两个变

量对间接散射强度进行估算^[38]。

4.4 体积云渲染的优化

近年来,不论是计算机的中央处理器,还是图形处理器,它们的计算速度都得到了非常大的发展,时钟频率高达上 GHz;另一方面,内存的发展则主要集中在存储密度而不是传输速度的增加上,这就导致了处理器速度的远远超过了内存速度的,不论处理器的速度多快,程序的运行效率都受限于内存速度这个瓶颈,这个瓶颈也称之为冯·诺依曼瓶颈^[42]。

在实时渲染阶段,由于冯·诺依曼瓶颈的存在,对于云层的光照计算可以在较短的时间内完成,耗时较长的指令主要是对纹理的采样。如果使用不经优化的算法直接对云层进行渲染,一个被云覆盖的像素点,每一帧的渲染最多可以进行多达上千次纹理采样,即便每次采样的数据都为 4 通道的 8 比特整型,也涉及到了至少 4MB 数据的读取。在高分辨率的情况下,这显然是不可接受的。

为此,需要对云层的渲染进行优化。本文的优化借鉴了时间混叠反走样技术 (Temporal Anti-Aliasing, TAA) 的思路。首先介绍 TAA 的原理。

3D 渲染的实质是对多个连续的函数,如场景的几何信息,双向反射分布函数 (Bidirectional Reflectance Distribution Function, BRDF) 进行采样,然后转化为一个离散的二维函数,即一定分辨率的图像的过程。奈奎斯特采样定理指出,这样的转化在分辨率不够高时必然会产生走样。在计算机图形学中,主要存在两大走样类型:一种是对空间域的信号采样频率不足导致的走样,称为空间走样,它包括几何走样,着色走样等;另一种走样则是由于帧率的限制,导致对相邻时间内的信号采样不足导致的走样,称为时间走样^[42]。TAA 技术就是一种时间反走样技术,它将原本在空间中的多次采样分摊到了多个时间帧中,如图 4-15 所示:

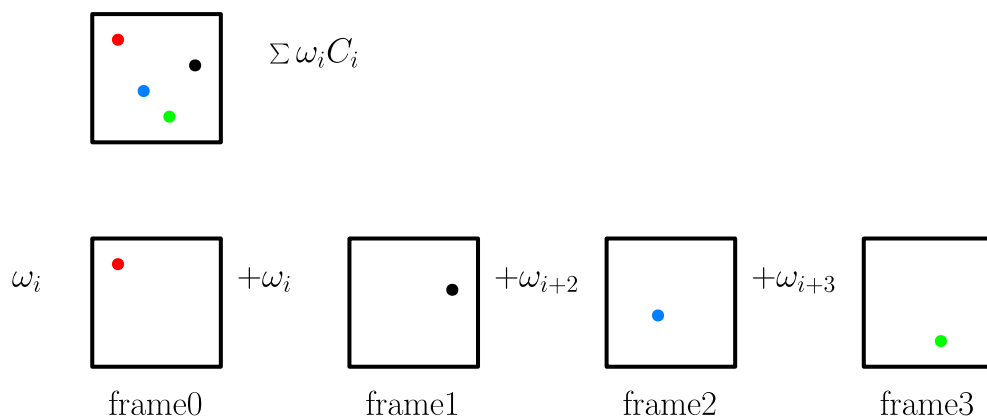


图 4-15 TAA 原理示意图^[42]

TAA 会将当前帧的信息, 计算当前像素对应的物体在历史帧中的位置, 从而使用历史帧的计算结果与当前值进行加权平均。为此, 需要计算每个当前像素相对于上一帧的偏移量, 成为移动矢量。当前像素点对应的物体在上一帧裁剪空间的位置的计算可以在顶点着色其中进行, 如公式 (4-18) 所示:

$$\begin{aligned} pos_{proj} &= pos_{obj} \times M_{mvp} \\ pos_{Last_{proj}} &= pos_{Last_{obj}} \times M_{Last_{mvp}} \end{aligned} \quad (4-18)$$

其中 pos_{obj} 和 $pos_{Last_{obj}}$ 分别为当前帧像素对应物体在当前帧和上一帧的局部坐标, M_{mvp} 和 $M_{Last_{mvp}}$ 分别为当前帧和上一帧的坐标变换矩阵。经过光栅化后, pos_{proj} 和 $pos_{Last_{proj}}$ 会变换到屏幕空间, 屏幕空间内像素的移动矢量就可以由公式 (4-19) 得到:

$$motionVector = pos_{Last_{screen}} - pos_{screen} \quad (4-19)$$

移动矢量的计算结果会被保存到纹理。有了移动矢量缓存就可以在着色阶段确定当前像素在上一帧中的位置, 从而在上一帧的颜色缓存中得到历史颜色值。

本文使用的优化方法与 TAA 的思路类似, 首先是颜色缓存划分为若干个 4×4 的像素块, 每一帧只对其中的一个像素更新体积云的渲染, 如图 4-16 所示:

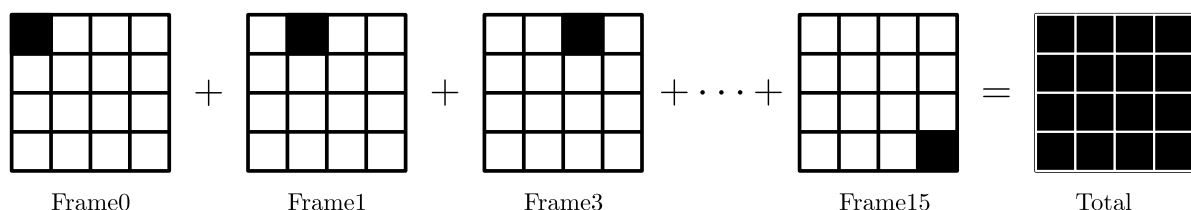


图 4-16 逐像素更新示意图

对于没有更新的像素, 由于体积云顶没有点数据, 因此需要直接使用像素以及相机的信息计算出像素在上一帧的位置, 具体算法如下: 首先将像素的坐标变换到裁剪空间, 之后使用当前帧相机的逆变换矩阵, 计算出世界空间下视线方向。在通过前一帧的相机变换矩阵计算在前一帧的像素坐标。如果坐标是有效坐标, 即当前帧的像素在上一帧也被标记为天空渲染, 则对上一帧的颜色缓存进行采样。如果是无效的坐标, 则对该像素进行体积云的渲染。

通过这种方法, 理论上能够将计算量减小到原先的 $\frac{1}{16}$, 极大地提升渲染的效率; 并且由于云的现状原本就基于随机的算法, 因此这样的间隔帧更新并不会影响到云层渲染的效果, 但与所有基于时间反走样的技术一样, 当视角快速变化时会在画面上产生拖影。

4.5 本章小结

本章主要介绍了 Perlin 噪声以及 Worley 噪声的生成，以及使用这两种噪声生成云朵形状和云层分布的方法，同时也对云层的光照模型进行了理论描述。由于体渲染至今仍是一种比较昂贵的渲染技术，本节也相应提出了一种基于时间反走样思路的体积云渲染优化技术，极大地提升了渲染的效率。

第5章 基于物理的大气实时渲染技术的设计与实现

5.1 渲染系统的总体架构

本文实现的基于物理的大气实时渲染系统的总体结构如图 5-1 所示。渲染系统主要负责两部分：数据更新和实时渲染。其中，数据更新部分负责计算、更新渲染所需的数据，并将这些数据提供给实时渲染的部分使用；实时渲染部分则根据数据更新的结果，依次对各个部分进行渲染。本章将基于该结构示意图对系统实现的各个部分进行阐述。

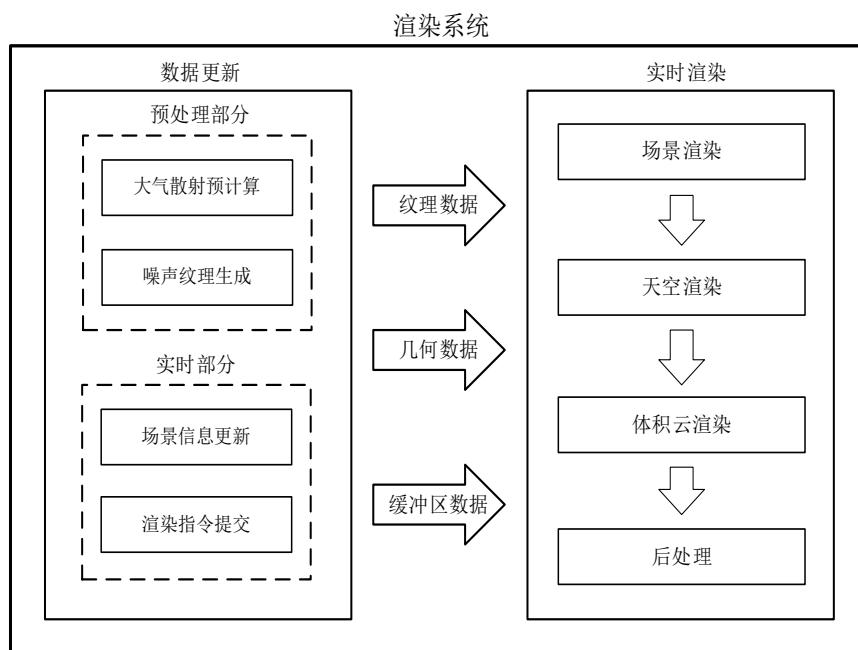


图 5-1 系统结构示意图

数据的更新可以分为实时更新部分和非实时更新部分，也就是预处理两个部分。预处理部分主要负责大气散射的预计算以及噪声纹理的生成。这些数据的特点是，计算的时间较长，并且设定好参数之后，计算的结果不会随着系统的更新而改变。因此，这些数据可以在渲染开始之前进行计算，并由渲染系统进行管理，提交由实时渲染部分使用。数据的实时更新部分则包括了场景信息更新以及渲染指令提交两个部分。场景信息更新主要包括了视点的更新，以及场景的加载与卸载，太阳高度角的变化等。由于使用 DirectX 12 作为系统的渲染 API，因此完成数据的更新后，还需要记录完成实时渲染所必须的渲染指令，并提交至实时渲染部分。

实时渲染部分则使用数据更新部分提供的纹理，几何以及常量数据，利用 GPU 的并行计算能力，依次完成场景，天空以及体积云的渲染，对渲染的结果进行后处理(Post-Processing)，以完成反走样，调色等效果。本章的主要内容将会围绕实时渲染部分展开。

5.2 基于 DirectX 12 的渲染流程

图形处理器(GPU)是 NVIDIA 公司在 1999 年 8 月发布 NVIDIA GeForce 256 绘图处理芯片时首先提出的概念^[43], 图形处理器在诞生之初, 其设计就为大规模的并行运算进行了优化, 拥有自己独立的计算单元和显存单元, 并通过 PCI-E 总线与 CPU 进行数据的交互。与 CPU 相同, 图形处理器也可以对计算单元进行编程, 完成特定的计算任务。在图形计算领域, 渲染 API 就为图形处理器的接口进行了封装, 以便开发人员使用图形处理器的各项功能。主流的渲染 API 有 OpenGL, DirectX, Vulkan, Metal 等, 其中 DirectX 是微软公司创建的一系列专为多媒体以及游戏开发的应用程序接口^[44], 相较于其他的渲染 API, 依托 Windows 系统在多媒体以及游戏方面的广泛应用, 以及微软公司的技术实力, DirectX 有着成熟, 新技术应用速度快的优势。其中 DirectX 12 是微软公司于 2015 年发布的最新一代的渲染 API, 与之前的各个版本的 DirectX 以及 OpenGL 不同, DirectX 12 专为当代的图形处理器进行了重新的设计, 提供了更为底层的硬件抽象, 使得开发人员能够在更细的粒度上对图形处理器的内存进行管理, 并且对渲染管线的各个流程进行调度, 从而提升渲染的效率。在图形处理器负载较高的时候, DirectX 12 相较于 DirectX 11, 平均能有至少 10%左右的性能提升^[45]。

由于本文所涉及的渲染算法较为复杂, 因此从实际性能表现的角度出发, 选择使用 DirectX 12 渲染 API 作为本文算法实现的渲染 API。运行在 CPU 中的程序通过渲染 API 调用图形处理器进行绘制的流程如图 5-2 所示:

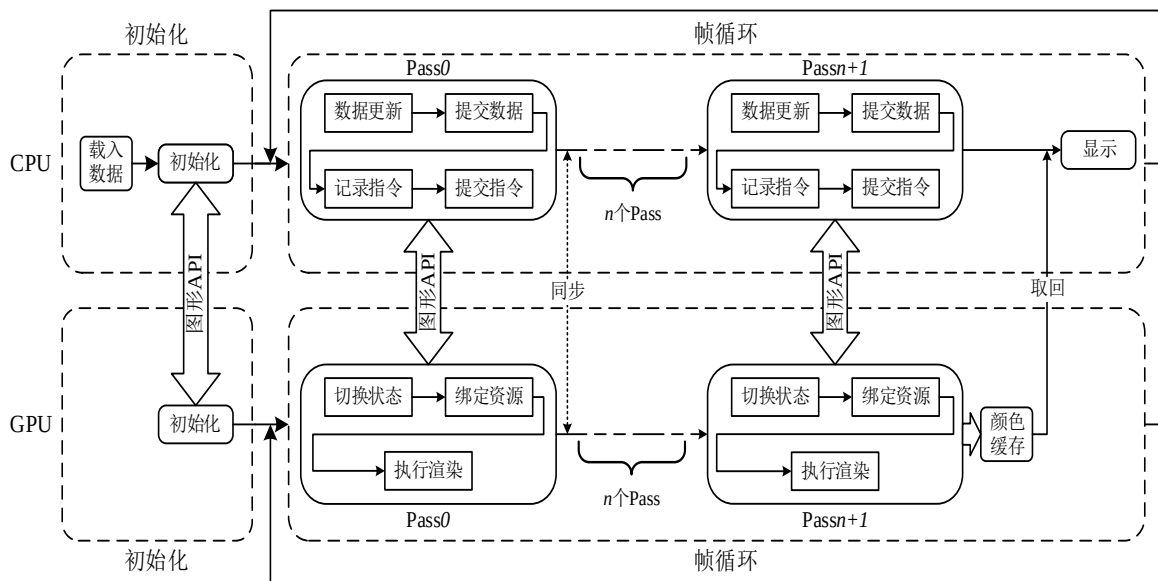


图 5-2 CPU 与 GPU 工作流程示意图

程序会在初始化阶段将所需要的资源,如纹理,几何数据等加载到内存中,然后通过渲染 API 创建对图形处理器硬件的抽象,以便后续创建图形处理器上相应的指令。在程序的帧循环阶段,程序会将一帧的完整绘制划分为多个 Pass,对于每个 Pass,程序会对 Pass 中所需数据进行更新,并将图形处理器所需要的部分数据通过渲染 API 从系统内存加载到图形处理器所使用的显存中。完成数据的准备后,程序会切换渲染状态,创建相应的绘图指令,指定绘图指令所需要的资源在显存上的地址,并将这些指令发送到图形处理器。硬件驱动会将这些渲染 API 的命令转换为图形处理器硬件可以执行的本地机器指令。完成一个 Pass 的渲染后,CPU 会与图形处理器可以根据实际的情况,选择进行资源的同步,或是直接进行下一个 Pass 的渲染。完成一个帧循环中所有 Pass 的渲染后,程序将最终的颜色缓存取回到系统内存中,并在显示设备上显示。

由图 5-2 可知,CPU 与图形处理器之间的工作是并行的,在图形处理器端,针对图形渲染的过程有着自己的渲染管线,如图 5-3 所示。渲染管线的意义是,如果给出一个三维场景的几何描述,并在其中设置一台具有确定位置以及视线朝向的虚拟相机,以该相机为观察视角生成二维图像的一系列完整步骤。计算所需的数据为几何数据,纹理数据以及缓冲区数据。其中几何数据为三维场景的几何描述,纹理数据和缓冲区数据则是着色器运行阶段所需要的数据。计算的过程具体为:在输入装配阶段,图形处理器会根据几何数据的描述符,从显存中读取几何数据并将其装配为诸如三角形、线条等构成图形的基本元素,称之为图元(Primitive)。完成这一阶段后,可以使用着色器对数据进行计算,着色器为在图形处理器上运行的程序,不同阶段的着色器对数据有着不同的处理方式。顶点着色器会对每个图元进行逐顶点的处理,将顶点的坐标变换到相机的投影空间。完成顶点的处理后,可以使用可选的几何着色器对每个图元进行处理,例如将点拓展成一个四边形等。完成图元的处理后,几何着色器可以将顶点数据流输出到显存的某个缓冲区中,等待下一步操作。光栅化阶段,图形处理器的硬件会对顶点数据进行裁剪,去掉相机中看不见的部分,然后根据图元顶点的属性以及图元在二维图像中的位置,对图元内部各个像素的属性进行逐像素插值。确定各个像素的属性之后,像素着色器就可以根据开发人员编写的程序,计算图元对应的各个像素的颜色值,并在计算完成后,在输出合并阶段将各个图元的颜色合并到最终的颜色缓存上,并将结果返回给显存,以便 CPU 取回并显示。

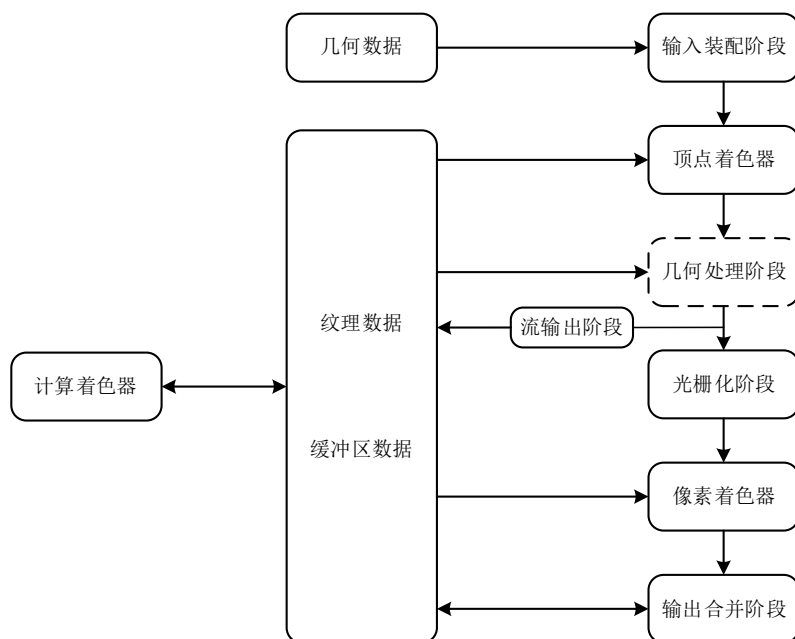


图 5-3 GPU 的渲染管线

在图 5-3 中，展示了另外一种独立于渲染管线存在的着色器，计算着色器。计算着色器是针对现代图形处理器的大规模并行处理架构而设计的一种着色器。以 NVIDIA 公司的图形处理器为例，处理器的基本调度单位被称为流式多处理器(Streaming multiprocessor, SM)，NVIDIA 公司的安培架构处理的 SM 架构如图 5-4 所示。一个 SM 中含有寄存器，共享内存，以及 32 个被称为流处理器(Streaming processor, SP)的基本处理单元。一个 SM 中的处理单元之间可以利用共享内存进行高速的数据共享与线程同步。计算着色器可以非常方便地将计算量分摊到图形处理器的多个 SM 中的每个 SP 上，以实现计算的并行加速。

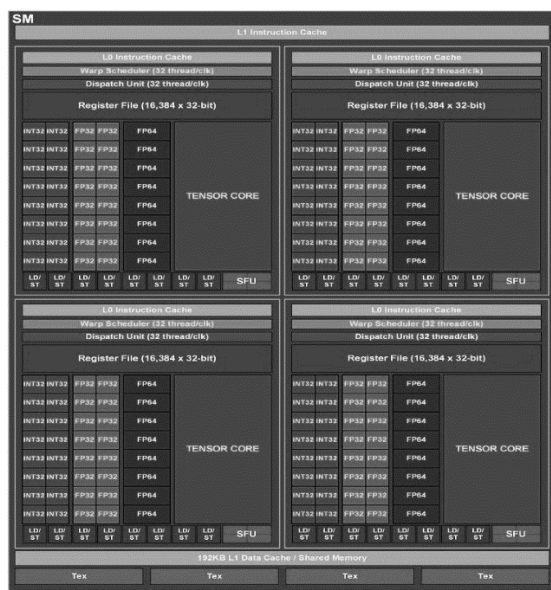


图 5-4 流式多处理器架构[46]

由于 DirectX 12 开放了 GPU 同步的控制接口，因此可以将计算着色器作为一个独立的单元，与 GPU 原有的渲染管线进行并行计算，达到性能的优化。计算着色器也可以实现一些原本由 GPU 渲染管线完成的功能。例如 Unreal Engine 5 中的 Nanite 技术，就使用计算着色器去执行原本由光栅化器执行的光栅化操作，以达到在场景的几何复杂度极高的情况下的性能优化。本文将利用计算着色器，实现天空以及体积云的渲染，以提升程序的性能。具体的利用方法就，是对于需要生成的图像的各个像素，都通过计算着色器，将像素颜色值的计算算法调度到 GPU 中对应的一个 SP 进行计算，达到像素间并行计算的效果，同时省去光栅化中的多余的操作，提高计算的效率。

5.3 大气渲染的实现

本节将介绍大气渲染的具体实现，以延迟渲染管线为例，大气渲染的流程图如图 5-5 所示。首先需要在准备阶段，对透射率以及内散射的辐射率进行预计算，将结果存储在纹理中。然后在实时渲染阶段对场景进行渲染，完成渲染之后，使用计算着色器，对帧缓存的每个像素进行计算，利用深度缓存判断像素对应的物体是否为天空，如果是场景中的物体，则根据物体的材质和场景属性，计算物体表面出射的辐射率。之后根据预计算的结果计算物体表面与视点之间的透射率和内散射辐射率，将其纳入像素最终颜色的计算。如果像素对应的是天空，则根据预计算的结果快速得到天空入射视点的辐射率，并由此计算天空的颜色。最终，完成像素颜色的计算后，将结果输出到帧缓存中，完成渲染。

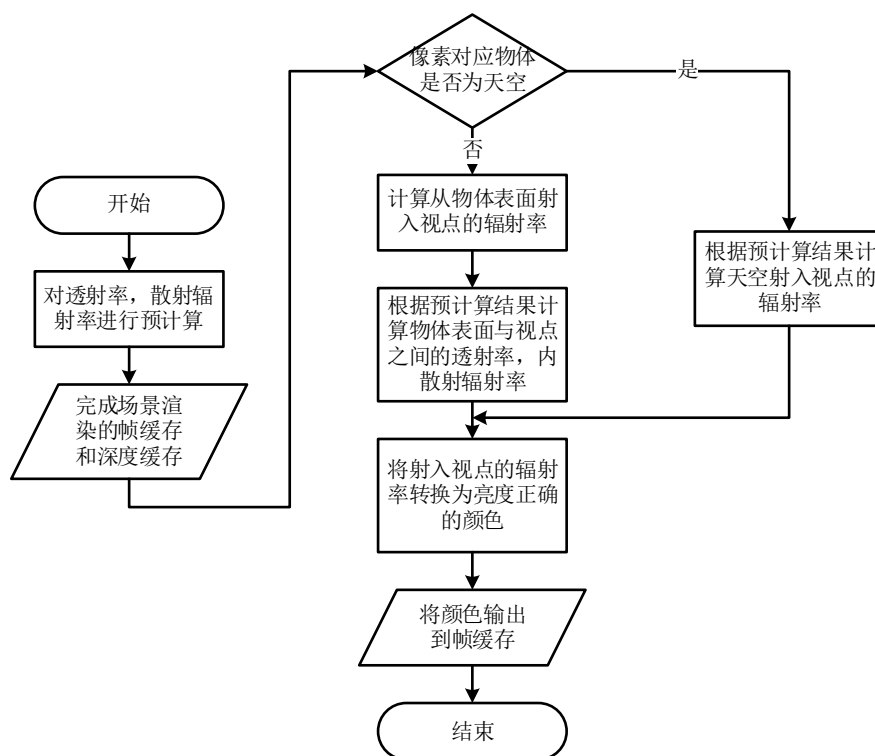


图 5-5 大气渲染流程图

本节将从预计算纹理的生成, 物体的环境光照计算以及天空辐射率的计算三个部分对大气渲染的实现展开进行描述。

5.3.1 预计算纹理的生成

纹理的数据结构非常类似于数组, 数组由一个个元素组成, 每个元素存储有数据, 元素又服从一维或者多维的排布顺序, 有着自己的坐标; 纹理则由一个个纹素(Texel)组成, 按照二维或者三维的顺序进行排布(一维纹理可以看作只有一行纹素的二维纹理)。与数组不同的是, 数组的访问不可越界, 对纹素的访问则可以通过对寻址模式的设置, 将越界部分的坐标按照一定的方式, 映射到正确的地址, 从而进行“越界”访问。另一方面, 读取纹素所存储的数据时, 可以通过对过滤器的设置, 返回与周围纹素数据插值的结果。为了将透射率, 内外散射的结果存储在纹理内, 需要建立纹素坐标与物理量之间的映射关系。

由 3.2 节可知, 视点到大气层顶部的透射率是关于视点高度 r 和射入视点的光线的天顶角 θ 两个变量的函数, 因此计算的结果可以存储在一张二维纹理中。由于不同视点高度对应的 θ 的范围不同, 并且在渲染时, 地平线附近的像素相比视点上方的像素对应了更大的范围, 因此出于精度以及实际计算的考虑, 映射的方式要将有效的 r 和 θ 值映射到纹理上, 并且令地平线附近的角对应纹理上更多的纹素。具体的映射方法如图

5-6 所示:

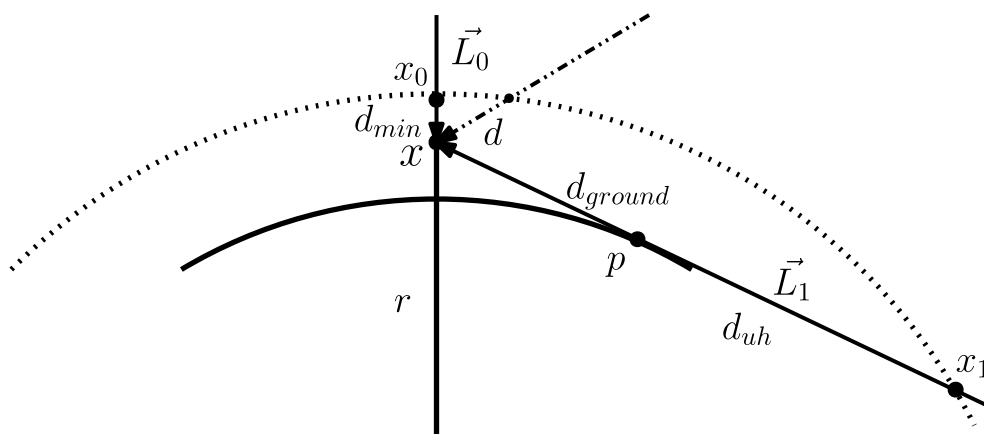


图 5-6 视点高度 r 和进入视点的光线天顶角 θ 映射示意图

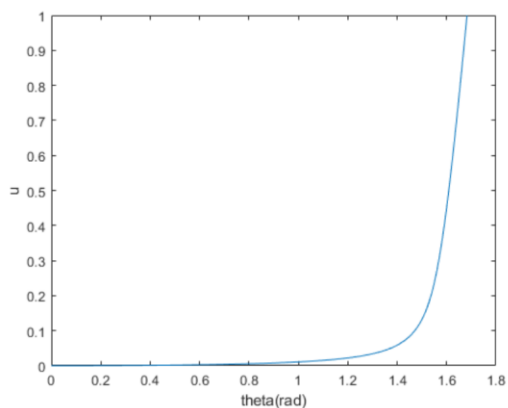
图中 $\vec{L}_0$ 和 $\vec{L}_1$ 分别代表视点 x 高度一定时, 进入视点的光线的天顶角 θ 的最小值和最大值, 其中, $\vec{L}_1$ 为沿地平线射入视点的光线。此时有视点与大气层边界距离的最小值 d_{min} 和最大值 $d_{max} = d_{ground} + d_{uh}$, 且随着 θ 的增加而递增。因此可以将对 θ 映射转化为对到大气层边界距离 d 的映射。具体的计算方式如公式(5-1)所示。

$$u = \frac{d - d_{min}}{d_{max} - d_{min}} \quad (5-1)$$

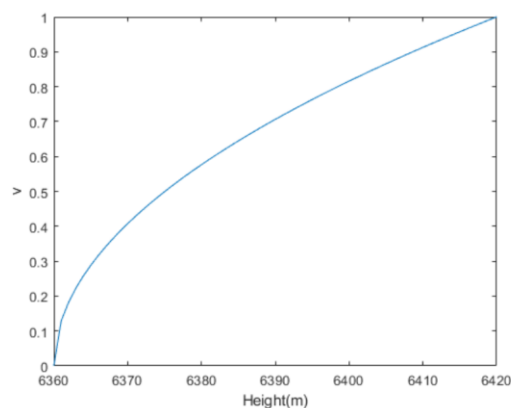
对视点高度 r 的映射可以使用均匀映射, 但由于大部分情况下视点都在地表附近, 因此需要提高在地表附近的存储精度, 映射方式如公式(5-2)所示:

$$v = \frac{d_{ground}}{d_{uh}} \quad (5-2)$$

变量 (θ, r) 和映射结果 (u, v) 之间的关系如图 5-7 所示, 可见在角度较大以及高度较低的区域, 计算的结果可以存储在更多的纹素中。



a) $\theta - u$ 的映射关系曲线



b) $r - v$ 的映射关系曲线

图 5-7 参数 (θ, r) 与纹理坐标 (u, v) 之间的映射关系示意图

有了纹素坐标的映射关系后,就可以对每个纹素,计算其对应的透射率的值,计算的伪代码如图 5-8 所示。

Algorithm: 透射率计算算法

Input: 纹素坐标 (u, v) , 大气层参数

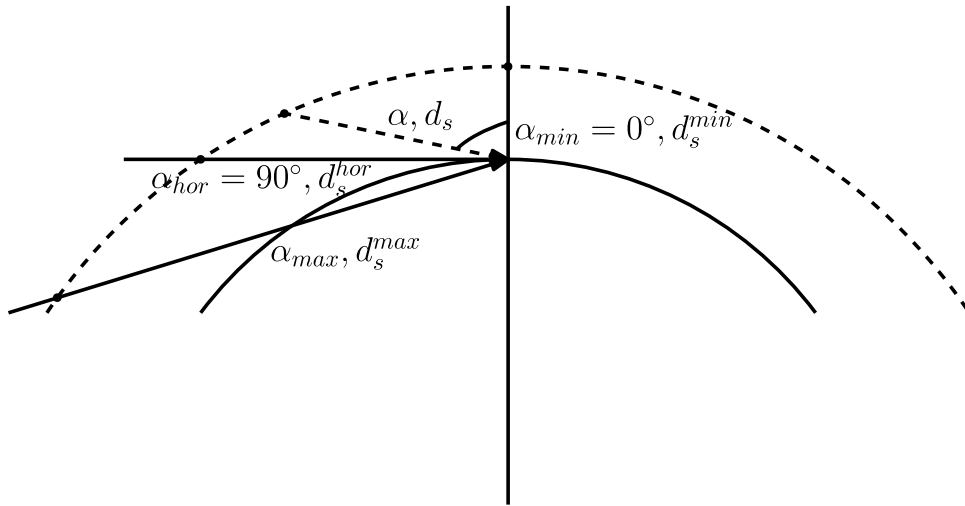
Output: 纹素对应的透射率 T

- 1 由纹素坐标 (u, v) 计算视点高度和进入视点的光线的天顶角 (r, θ) ;
- 2 $d \leftarrow$ 由 r 和 θ 计算出视点沿 θ 方向到大气层边界的距离;
- 3 $N \leftarrow$ 采样次数;
- 4 $dx = \frac{d}{N}$;
- 5 $i \leftarrow 0$;
- 6 $l \leftarrow 0$;
- 7 **for** $i \leq N$ **do**
- 8 $d_i \leftarrow idx$;
- 9 $r_i \leftarrow \sqrt{d_i^2 + 2r \cos(\theta)d_i + r^2}$ // 计算采样点的高度;
- 10 $\sigma_a \leftarrow D_R(r_i) + D_M(r_i) + D_E(r_i)$ // 计算 r_i 高度下的透射系数;
- 11 $w \leftarrow (i = 0 \parallel i = N) ? 0.5 : 1.0$ // 梯形公式;
- 12 $l \leftarrow l + \sigma_a w dx$ // 累计光学深度;
- 13 **end**
- 14 $T \leftarrow e^{-l}$ // 计算透射率;
- 15 **return** T

图 5-8 透射率计算伪代码

由 3.3.1 节可知,直接散射与间接散射的计算结果是关于视点高度 r ,进入视点的光线的天顶角 θ 以及太阳天顶角 α 三个变量的函数,因此计算的结果可以存储在三维纹理中。其中视点高度 r 的映射与透射率计算过程中视点高度 r 的映射相同;天顶角 θ 的映射与透射率计算中的映射相同,但由于散射不仅需要计算视点到大气层边界之间的散射结果,还需要计算视点到地面的散射值,因此 d_{max} 的计算需要分为两部分,当光线由天空进入视点时, $d_{max} = d_{ground} + d_{uh}$,当光线由地表进入视点时, $d_{max} = d_{ground}$ 。

太阳天顶角 α 的示意图如图 5-9 所示,由于不同的视点高度对应的 α 的范围不同,当视点在地表时,对应的 α 范围最小,此时 $\alpha \in [0^\circ, 90^\circ]$ 。并且由于大气散射的存在,太阳处于地平线以下时,也可以接受到来自太阳的间接散射,因此在地平线以下的部分也应该被计算和记录在纹理中。

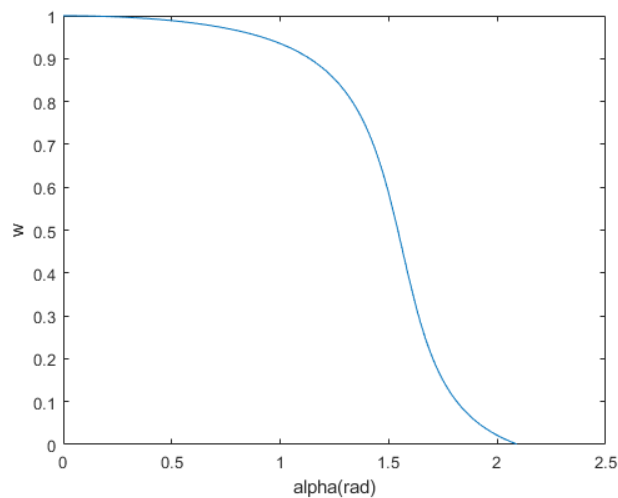
图 5-9 太阳天顶角 α 范围示意图

为了增加地平线附近的精度，可以选用与角度 θ 类似的映射方式，将角度的映射转化为距离的映射。选定地表的视点为计算点，将 $\alpha > 90^\circ$ 的部分加以放大。图中， d_s 为从地表出发，沿太阳光到大气层边界的距离， α_{max} 为最大的太阳天顶角， α 到纹理坐标的映射计算如公式（5-3）所示：

$$A = \frac{d - d_s^{min}}{d_s^{hor} - d_s^{min}} \quad (5-3)$$

$$w = \frac{d_s^{max} - d}{d_s^{max} - d_s^{min}} \cdot \frac{1}{1 + A}$$

角度 α 和纹理坐标 w 之间的映射关系如图 5-10 所示，由式（5-3）可知， α 处于最大和最小值时，分别对应纹理坐标的 0 和 1，且当 α 在 90° 附近时占据了更多的纹素，拥有更好的精度。

图 5-10 太阳天顶角 α 与纹理坐标 w 之间的映射关系图

得到纹理坐标与物理参数之前的关系之后，就可以对三维纹理的每个像素，计算其

像素对应的物理参数, 并进行相应的散射计算。直接散射的计算伪代码如图 5-11 所示。

Algorithm: 直接散射计算算法

Input: 纹素坐标(u, v, w), 大气层参数, 透射率预计算纹理 T , 太阳辐照度 L_{sun}

Output: 纹素对应的直接散射辐射率 S_0

- 1 由纹素坐标(u, v, w)计算相应的参数(r, θ, α);
- 2 $d \leftarrow$ 由 r 和 θ 计算出视点沿 θ 方向到大气层边界或者地表的距离;
- 3 $N \leftarrow$ 采样次数;
- 4 $dx = \frac{d}{N}$;
- 5 $i \leftarrow 0$;
- 6 $S_0 \leftarrow 0$;
- 7 **for** $i \leq N$ **do**
- 8 $d_i \leftarrow i \cdot dx$;
- 9 $r_i \leftarrow \sqrt{d_i^2 + 2r \cos(\theta)d_i + r^2}$ // 计算采样点的高度;
- 10 $\alpha_i \leftarrow \arccos(\frac{r \cos(\alpha) + d_i \cos(\theta + \alpha)}{r_i})$ // 计算采样点处的太阳天顶角;
- 11 根据 r, θ, d_i 获取视点与采样点之间的透射率 $T(r, r_i)$;
- 12 根据采样点处的太阳天顶角 α_i 获取采样点到太阳之间的透射率 $T(r_i, sun)$;
- 13 $T \leftarrow T(r, r_i)T(r_i, sun)$;
- 14 $\sigma_s = D_R(r_i) + D_M(r_i)$ // 根据 r_i 和太阳天顶角计算;
- 15 $w \leftarrow (i = 0 \parallel i = N) ? 0.5 : 1.0$ // 梯形公式;
- 16 $S_0 \leftarrow S_0 + \sigma_s T w dx$ // 累计光学深度;
- 17 **end**
- 18 $S_0 \leftarrow S_0 \cdot L_{sun}$
- 19 计算散射后的辐射率;
- 20 **return** S_0

图 5-11 直接散射计算伪代码

间接散射的计算变量与直接散射相同, 因此可以使用与直接散射相同的结果存储方式。与直接散射不同的是, 每一级的散射的计算都需要分为两步: 第一步, 计算公式(3-13)的球面积分; 第二步, 计算公式(3-14)所示的线积分。间接散射辐射率的计算的第一步的伪代码如图 5-12 所示。

Algorithm: 第 n 级散射计算的第一步

Input: 纹素坐标 (u, v, w) , 大气层参数, 透射率预计算纹理 T ,
第 $n-1$ 级散射预计算纹理 $L_{S_{n-1}}$

Output: 纹素对应的点接受第 $n-1$ 级散射后出射的辐射率 L'_{S_n}

```

1 由纹素坐标 $(u, v, w)$ 计算相应的参数 $(r, \theta, \alpha)$ ;
2 计算第 $n$ 级散射进入视点的方向 $\vec{\omega}$ ;
3  $N \leftarrow$  采样次数;

4  $d\phi \leftarrow \frac{\pi}{N}, d\varphi \leftarrow \frac{\pi}{N}$ ;
5  $L'_{S_n} \leftarrow 0$ ;
6  $i \leftarrow 0$ ;
7 for  $i \leq N$  do
8    $\varphi \leftarrow i \cdot d\varphi$ ;
9    $j \leftarrow 0$ ;
10  for  $j \leq 2N$  do
11     $\phi \leftarrow j \cdot d\phi$ ;
12    由 $\phi, \varphi$ 计算第 $n-1$ 级散射入射采样点的光线方向 $\vec{\omega}_{ij}$ ;
13     $d\omega_{ij} \leftarrow d\phi d\varphi \sin(\varphi)$ ;
14     $L_{S_{n-1}} \leftarrow S_{n-1}(r, \vec{\omega}_{ij}, \alpha)$  // 根据上一次计算的结果和当前参数
      查询第 $n-1$ 级散射沿 $\vec{\omega}_{ij}$ 入射的辐射率 $L_{ij}$ ;
15     $\cos(\nu) \leftarrow \vec{\omega} \cdot \vec{\omega}_{ij}$  // 计算入射方向和散射方向夹角的余弦值;
16     $\sigma_R \leftarrow D_R(r), \sigma_M \leftarrow D_M(r)$  // 获取点 $r$ 处瑞利散射和米氏散
      射系数;
17     $L'_{S_n} \leftarrow L'_{S_n} + L_{ij} \cdot (\sigma_R P_R(\cos(\nu)) + \sigma_M P_M(\cos(\nu))) \cdot d\omega_{ij}$  //
      计算散射后进入视点的辐射率,  $P_{R,M}(\cos(\nu))$ 为相函数;
18  end
19 end
20 return  $L'_{S_n}$ 

```

图 5-12 间接散射辐射率第一步计算伪代码

第一步会根据纹素的坐标计算出纹素坐标对应的点在大气中的位置, 太阳光的入射角, 以及第 n 级散射的方向, 根据这些参数以及上一级的散射的计算结果, 对点收到的 $n-1$ 级散射进行球面积分, 计算向纹素对应的角度出射的辐射率, 记录在一张临时的纹理中, 供计算的第二步使用。

计算的第二步的伪代码如图 5-13 所示:

Algorithm: 第 n 级散射计算的第二步

Input: 纹素坐标 (u, v, w) , 大气层参数, 透射率预计算纹理 T ,
第 $n-1$ 级散射预计算纹理 $L_{S_{n-1}}$, 第 n 级散射计算的第一步的
计算结果 $L'_{S_{n-1}}$

Output: 纹素对应的第 n 级散射辐射率 L_{S_n}

```

1 由纹素坐标 $(u, v, w)$ 计算相应的参数 $(r, \theta, \alpha)$ ;
2  $d \leftarrow$  由 $r$ 和 $\theta$ 计算出视点沿 $\theta$ 方向到大气层边界的距离;
3  $M \leftarrow$  采样次数;
4  $dx = \frac{d}{M}$ ;
5  $k \leftarrow 0$ ;
6 for  $k \leq M$  do
7    $d_k \leftarrow k \cdot dx$  // 更新采样点;
8    $r_k \leftarrow \sqrt{d_k^2 + 2r \cos(\theta)d_k + r^2}$  // 计算采样点的高度;
9    $\alpha_k \leftarrow \arccos\left(\frac{r \cos(\alpha) + d_k \cos(\theta + \alpha)}{r_k}\right)$  // 计算采样点处的太阳天
      顶角;
10  根据 $r_k, \alpha_k, d_k$ 从第一步计算的结果中查询对应
      的 $L'_{S_{n-1}}(r_k, \alpha_k, d_k)$ ;
11  查询视点和采样点之间的透射率 $T(r, r_k)$ ;
12   $w \leftarrow (k = 0 \parallel k = N) ? 0.5 : 1.0$  // 梯形公式;
13   $L_{S_n} \leftarrow L_{S_n} + T(r, r_k)L'_{S_{n-1}}(r_k, \alpha_k, d_k)w dx$  // 线积分;
14 end
15  $L_{S_n} \leftarrow L_{S_n} + L_{S_{n-1}}$  // 将结果累计入纹理;
16 return  $L_{S_n}$ 

```

图 5-13 间接散射辐射率第 n 步计算伪代码

计算的第二步会根据纹素对应的位置和角度, 计算最终的第 n 级间接散射的计算结果, 并将结果累计进入纹理, 便于实时渲染时直接查询到前 n 级散射的计算结果的总和。

5.3.2 物体的环境光照计算

物体最终颜色不仅受到场景中光源的影响, 还会受到来自太阳的直接光照以及视点和物体之间的内散射的影响。在对透射率和散射进行预计算后, 就可以在实时渲染阶段查询到相应的结果, 提升渲染的效率。物体颜色的计算如公式(5-4)所示:

$$C(p_o) = \lambda \left(M \left(\sum L_i(p_o) + L_{sun} T(p_o, p_{sun}) \right) + L_{in-scatter}(p, p_o) P(v) \right) \quad (5-4)$$

其中, C 为物体表面点 p_o 最终的颜色, M 为物体的表面反射方程, $L_i(p_o)$ 的和为场景中光源在物体表面辐射率的之和, 在本文中不做考虑。 L_{sun} 为太阳的辐射率, $T(p_o, p_{sun})$ 为太阳和 p_o 之间的透射率。 $L_{in-scatter}(p, p_o)$ 为视点 p 和 p_o 之间由大气的内散射贡献的辐

射率, $P(v)$ 为散射的相函数。最终入射视点的辐射率总和经过转换函数 λ 转换后变为亮度正确的颜色值输出。

$T(p_o, p_{sun})$ 可以通过对透射率纹理使用 p_o 进行一次纹理采样得到, 内散射的计算如图 5-14 所示:

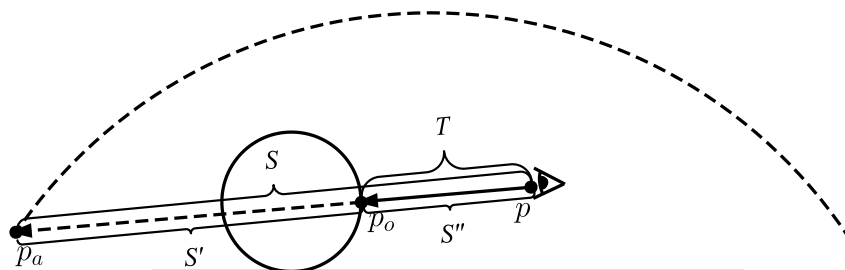


图 5-14 内散射计算示意图

图中, p 为视点, p_o 为物体表面一点, p_a 为视线延长线与大气层边界相交的点, 要计算的内散射值为 S'' 。由于上一节中, 预计算并保存在纹理中的内散射辐射率均为大气层边界到大气中一点的内散射辐射率, 因此不能直接通过查找得到 S'' 的值。将视线延长交大气层边界 p_a , 就可以通过纹理采样得到 p, p_o 与大气层边界之间的内散射辐射率 S, S' , S, S', S'' 之间的关系可以由公式 (5-5) 给出:

$$S = S'T + S'' \quad (5-5)$$

其中 T 为 p 与 p_o 之间的透射率, 由此可以通过两次纹理采样计算得到 S'' 的值。

5.3.3 天空颜色的计算

对于渲染天空的像素, 具体的计算公式可由公式 (5-6) 表示:

$$C(p_{sky}) = \lambda(L_{in-scatter}(p, p_a)P(v)) \quad (5-6)$$

其中 $\lambda(L)$ 色彩转换函数, $P(v)$ 为散射相函数, $L_{in-scatter}(p, p_a)$ 为视点到大气层边界的内散射辐射率, 该值可以直接从预计算纹理中访问得到, 因此天空颜色的计算只需要一次纹理采样即可得到。

对于天空中太阳颜色的计算, 可以设定一个太阳的角半径 δ , 当像素对应的视线方向与太阳光方向的夹角小于角半径时 δ , 就对该像素进行太阳的渲染。计算方式如公式 (5-7) 所示:

$$C(p_{sun}) = C(p_{sky}) + \lambda(L_{sun}T(p, p_a)) \quad (5-7)$$

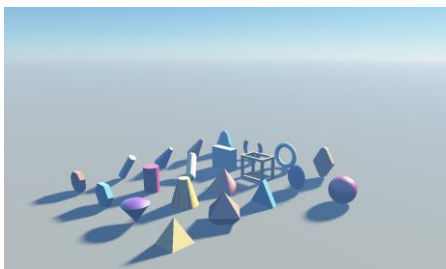
其中 L_{sun} 为太阳光在大气层边界处的辐射率, $T(p, p_a)$ 为视点到大气层边界的辐射率, 可以通过对预计算纹理的采样得到。由于看向太阳的视线方向上, 仍旧会收到来自天空的内散射的影响, 因此需要将之前计算的天空颜色叠加到像素的最终颜色中。由

于,透射率在预计算考虑了光线入射角度的影响,因此可以渲染出夕阳的效果。夕阳的渲染效果如图 5-14 所示。

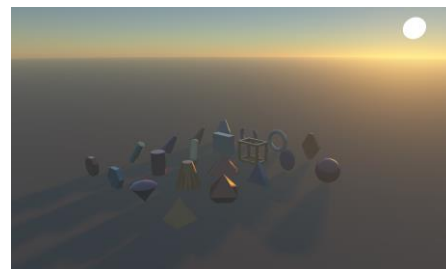


图 5-15 夕阳的渲染效果

场景中物体与天空在不同时间段的渲染结果如图 5-15 所示。



a) 白天时的渲染结果



b) 黄昏时的渲染结果

图 5-16 物体与天空的渲染结果

5.4 体积云渲染的实现

本节将介绍体积云渲染的具体实现,主要从以下几个方面展开进行描述:噪声纹理的生成,空间中体积云密度的确定,体积云光照的实现,体积云的体渲染技术以及体积云的渲染优化。体积云的渲染流程图如图 5-17 所示:

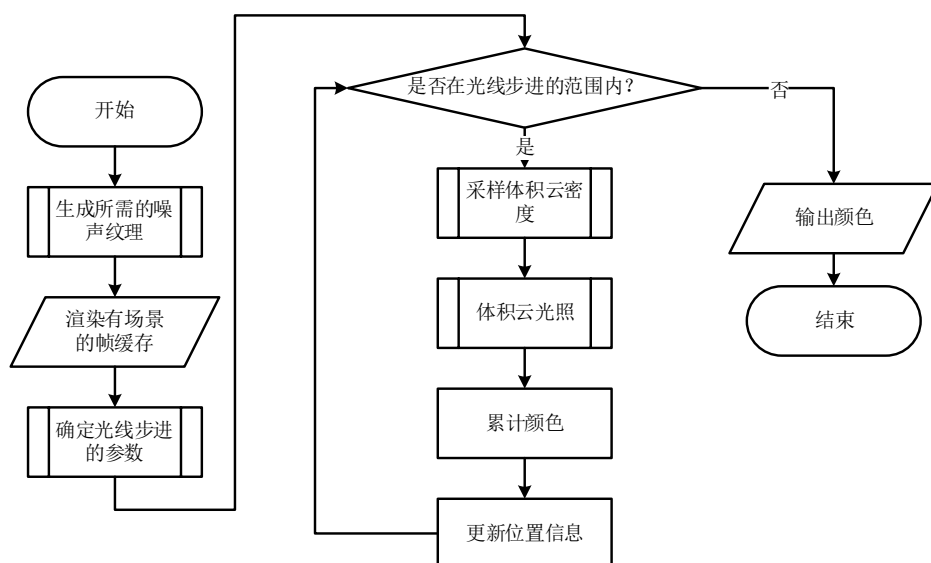


图 5-17 体积云渲染流程图

5.4.1 体积云的体渲染技术

体积云的渲染需要用到一种被称为光线步进(Ray-March)的体渲染技术, 光线步进的渲染原理示意图如图 5-18 所示。光线步进是一种对于场景中使用有向距离场函数描述的物体进行渲染的技术。所谓有向距离场函数, 就是指对于用有向距离场函数描述的物体, 在物体的内部的各点, 函数的值均大于零; 在物体的外部的各点, 函数的值均小于零; 而在物体表面的各点, 函数的值均为零。利用这个特点, 光线步进对于帧缓存的每个像素, 计算像素对应的视线在世界空间内的方向, 并将视线划分为一个个小段, 每段计算其世界空间内的坐标。将坐标代入有向距离场函数, 判断是否在物体表面或者内部。若是则进行相应的渲染, 若不是则继续步进, 直到达到预设的远平面或者视线与不透明的相交。

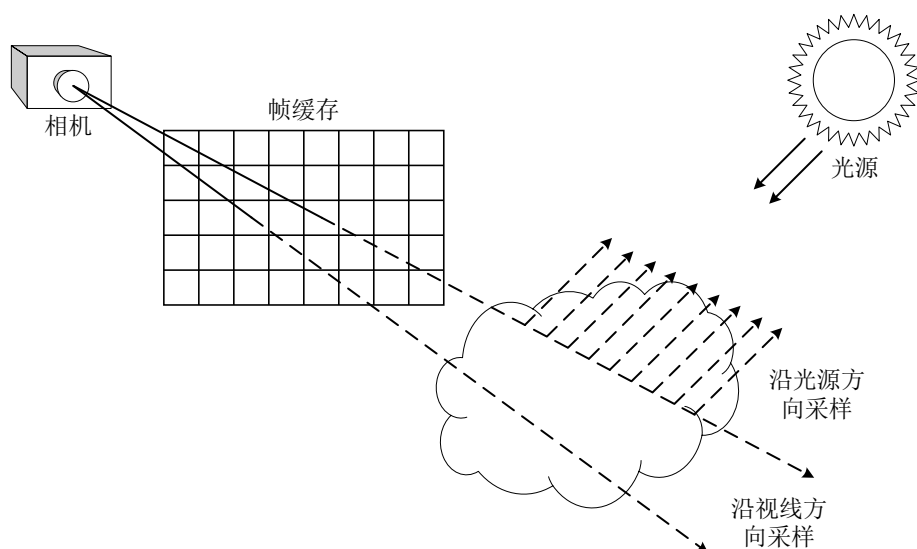


图 5-18 光线步进原理示意图

体积云的描述函数可以看作一种特殊的有向距离场函数, 对于在云层内部的点, 描述函数都会返回云层在该点的密度 (大于零), 对于云层没有出现的点, 则返回零。因此可以使用光线步进技术进行渲染。由于组成云的微粒并非不透明的实体, 因此即便光线步进采样到了云的内部, 也需要继续进行步进, 直到云层厚度达到一定的阈值。而对于云层内部的每个采样点, 都需要向光源方向进行步进采样, 以获取光源方向上的密度信息, 计算云层的光照。计算过程可由公式 (5-8) 表示。其中 $D(p)$ 为体积云的密度采样函数, $T(p)$ 计算了点 p 处的透射率, $S(p)$ 计算了点 p 处的光照。

$$L = \sum_{i=0}^N T(D(p_i)) S \left(\sum_j^M D(p_{ij}) \right) \quad (5-8)$$

体积云的渲染发生在场景、天空的渲染之后，后处理的流程之前，场景的颜色缓存和深度缓存会作为体积云渲染的输入，体积云则直接在颜色缓存上进行绘制。首先根据深度缓存判断像素对应的视线终点是否为天空，如果不是则直接跳出渲染流程；如果需要渲染体积云，则通过相机的参数，计算出像素对应的视线在世界空间内的方向。得到视线方向后根据云层出现的海拔范围，计算步进次数，并将光线步进的起始位置设为视线与云层底部的交界处。由于地球是一个球体，如图 5-19 所示。因此不同视线方向穿过云层分布区间的距离不同，如果采用固定的步进次数会导致在地平线附近的云层步长过长而引起走样。因此需要根据视线的方向，在地平线附近适当地增加光线步进的次数。

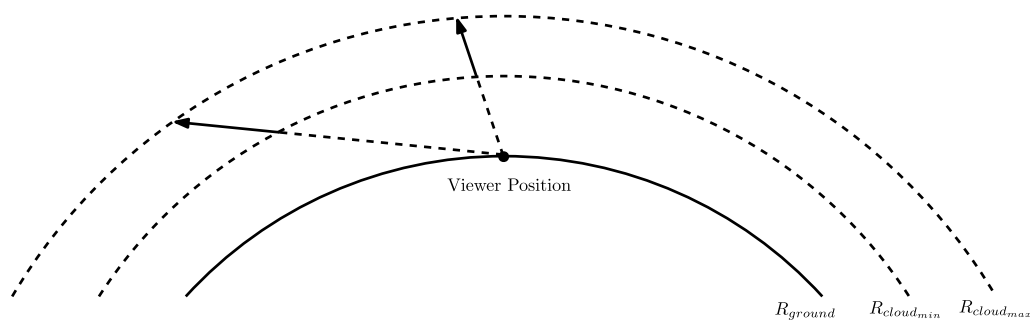


图 5-19 视线方向与步进总长的关系示意图

在计算光线步进的起始位置时，如果直接使用视线方向与云层底部半球面的交点，那么就相当于各个像素的采样函数均为单一的采样频率，导致采样的频带过低，会引起最终生成图像的严重走样，如图 5-20a 所示。为了避免这样的走样，需要在计算起始位置时引入一个随机值，对起始位置进行随机的偏置，得到较好的最终结果，如图 5-20b 所示。



a) 走样的结果示意图



b) 改进后的结果示意图

图 5-20 光线步进起始点引起的走样以及改进结果示意图

完成光线步进的初始化后，就沿视线方向，从光线步进的起始点向终点迭代计算采样点的位置。然后使用采样点的坐标，向噪声纹理查询云层的密度。如果该处的密度大

于零，则计算该点处的光照，并将得到的结果进行累计。最终得到体积云的渲染的结果。

5.4.2 体积云渲染所使用的纹理的生成

4.1 节中介绍了 Perlin 噪声与 Worley 噪声的生成方法，本节将基于 Guerrilla 公司在 SIGGRAPH 2015 与 SIGGRAPH 2017 中提出的理论^{[14],[38]}，介绍体积云渲染中所使用的各个噪声纹理的生成方法。

首先需要生成的是描述云层基本形状的噪声纹理。Perlin 噪声由于其特殊的性质，可以用于模拟一些较为薄的云层。对于那些较厚，并且具有翻滚形状的云，Perlin 噪声没有能够提供这种细节的形态，因此可以引入形态较为类似的 Worley 噪声来确定云层的基本形状。两种噪声均使用其分形噪声来提供更多的细节。Perlin-Worley 噪声的计算方法已由公式（4-6）给出。

由于纹理的一个像素最多可以存储 RGBA 四个值，为了节省渲染状态切换的开销，Perlin-Worley 噪声的值可以保存在纹理的一个通道内，其余的三个通道可以保存三个频率递增的 Worley 噪声的值，以提供云层以水汽翻滚的形状。如此生成的纹理称为形状纹理，分辨率为 $128 \times 128 \times 128$ ，图 5-21 展示了该纹理的一个切面上，四个通道内分别存储的值。

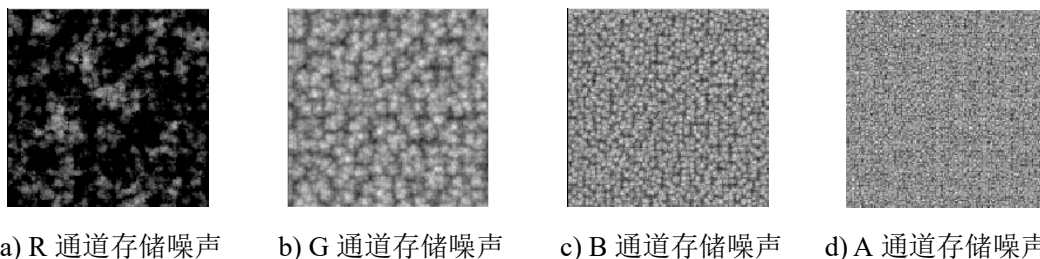


图 5-21 形状纹理各通道存储噪声示意图

该纹理只能提供体积云的基本形状，为了能够达到较为逼真的云层效果，还需要一张分辨率较低($32 \times 32 \times 32$)，存储有高频 Worley 噪声的纹理，用以在基本形状上提供更多的细节。低分辨率纹理一个切面上三个通道内存储的 Worley 噪声如图 5-22 所示。

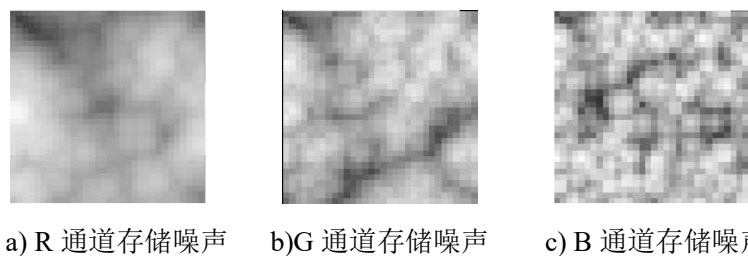


图 5-22 低分辨率纹理示意图

5.4.3 体积云密度的采样

第四章中, 公式(4-8)给出了体积云的密度函数 $shape(p)$, 本节将给出确定云层密度的具体方法。由于本文会将体积云的分布限制在一个确定的高度范围内, 因此首先需要将空间中某一点的高度进行归一化, 并将值限制在 $[0,1]$ 的范围内。计算归一化高度值的伪代码如图 5-23 所示。

Algorithm: 归一化高度值计算

Input: 空间坐标 p , 云层高度最大值 h_{max} 和最小值 h_{min}

Output: 归一化高度值 h

$h \leftarrow \frac{p.y - h_{min}}{h_{max} - h_{min}}$ // 将高度值归一化;

$h \leftarrow \max(\min(h, 1), 0)$ // 将值限制在 $[0, 1]$ 范围内;

return h

图 5-23 归一化高度值计算伪代码

计算得到归一化高度值后, 就可以利用 5.4.1 节中生成的云层形状纹理来确定云层的基本形状。对于空间中的一点 p , 将 p 的坐标值进行缩放后, 使用归一化高度值代替原本 p 点的 y 轴坐标, 然后对形状纹理 T_b 进行采样。采样得到的值 N 包含四个值, 其中 N 的 r 通道为 Perlin-Worley 噪声值, gba 通道为频率递增的 Worley 噪声值。将 gba 的三个值进行加权叠加后, 与 Perlin-Worley 噪声进行混合, 即可得到云层的基本形状。具体的计算伪代码如图 5-24 所示。

Algorithm: 云层基本形状

Input: 形状纹理 T_b , 坐标 p , 归一化高度值 h

Output: 点 p 处的云层密度 d

1 $p \leftarrow p \cdot S$ // 将坐标进行缩放;

2 $p.y \leftarrow h$ // 用归一化高度值替代坐标的高度值;

3 $N \leftarrow T_b(p).rgba$ // 使用缩放后的坐标值查询Perlin-Worley和低频Worley噪声值;

4 $d \leftarrow N.gba \cdot (0.625, 0.25, 0.125)$ // 将不同频率噪声值叠加;

5 $d \leftarrow \frac{N.r - d + 1}{2 - d}$ // 利用混叠的噪声计算 p 点处的密度;

6 $d \leftarrow \max(1, \min(0, d))$ // 将密度值限制在 $[0, 1]$ 范围内;

7 **return** d

图 5-24 云层基本形状确定伪代码

计算返回的结果 d 为点 p 处云的密度, 但此时的云层缺乏基本形状, 如果不加处理直接使用, 难以表现云层变化多端的特征, 如图 5-26a 所示。因此为了表现出更多的云层细节, 可以利用 5.4.1 节中的低分辨率 Worley 噪声纹理以及已有的云的密度 d 进行计

算，得到具有形状细节的云层，具体的计算伪代码如图 5-25 所示。可以将这个过程理解为在已有的云层基础形状基础上，利用高频 Worley 噪声的特征，“侵蚀”出具有细节层次的体积云形态。

Algorithm: 云层形状细节生成

Input: 低分辨率Worley噪声纹理 T_e , 坐标 p , 点 p 处云层密度 d_o

Output: 点 p 处的云层密度 d

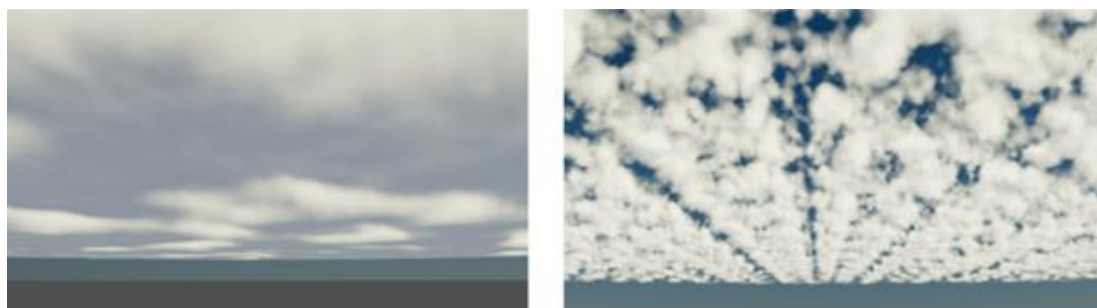
```

1  $p \leftarrow p \cdot S'$  // 使用不同的缩放因子对坐标进行缩放;
2  $p.y \leftarrow h$  // 用归一化高度值替代坐标的高度值;
3  $N \leftarrow T_e(p).rgb$  // 使用缩放后的坐标值查询高频Worley噪声值;
4  $d \leftarrow N.rgb \cdot (0.625, 0.25, 0.125)$  // 将不同频率的噪声值叠加;
5  $h \leftarrow \max(1, \min(0, h \times 10))$ ;
6  $m \leftarrow (1 - h)d + (1.0 - d)$ ;
7  $d \leftarrow d_o - m \times (1 - d_o)$ ;
8  $d \leftarrow \frac{2d - 0.2m}{1 - 0.2m}$  // 对云层的基本形状进行修改, 计算增加细节后的
   云层密度;
9 return  $d$ 

```

图 5-25 云层形状细节生成伪代码

完成处理后，就可以得到如图 5-26b 所示的具有显著的云层特征的云层形状。



a) 云层的基本形状

b) 调整后的云层形状

图 5-26 云层的基本形状与调整后的云层形状示意图

尽管得到了云层的形状,但如图 5-26b 所示,可以看到云层的排布有着明显的规律,不符合现实的情况,因此需要引入 4.2 节中所描述的云层水平分布纹理和形态函数,来生成具有不规则排布的云层。云层排布生成的伪代码如图 5-27 所示。

Algorithm: 云层排布生成

Input: 水平分布纹理 T_w , 坐标 p , 点 p 处云层密度 d_o , 天空云层的覆盖率 $a\%$

Output: 点 p 处的云层密度 d

```

1  $w = T_w(p.xz).rg$  // 使用水平坐标查询云层水平分布纹理;
2  $c \leftarrow w.r \times a\%$  // 计算覆盖系数;
3  $d \leftarrow \frac{d_o - c}{1 - c}$  // 使用覆盖系数修饰云层密度;
4  $d \leftarrow d \times c \times type(w.g)$  // 使用云层形状函数修饰云层密度;
5 return  $d$ 

```

图 5-27 云层排布生成伪代码

其中, 为了对天空中云层的覆盖率进行控制, 引入了变量 $a\%$ 加入计算。水平分布纹理 T_w 是一张二维纹理, 且只含有 RG 两个通道, R 通道包含了云层的分布图, G 通道为云层的类型。伪代码第 4 行中的函数 $type(x)$ 为公式 (4-10) 所示的云层类型密度函数。不同覆盖率的云层分布如图 5-28 所示。



a) 覆盖率较高的云层渲染结果



b) 覆盖率较低的天空云层渲染结果

图 5-28 不同覆盖率的天空云层示意图

为了模拟云层的运动, 可以在对低分辨率 Worley 噪声纹理 T_e 以及云层水平分布纹理 T_w 进行采样时, 对采样坐标进行按时间以及风速进行偏置, 以达到云层运动的效果, 计算的伪代码如图 5-29 所示。

Algorithm: 动态云层模拟

Input: 坐标 p , 时间 t , 风速 v , 风向 $\vec{w}$, 归一化高度值 h

Output: 偏置后的密度采样坐标 p_d

```

1  $a \leftarrow h * \vec{w} * S + \vec{w} * t * v$  // 使用风速, 风向和时间计算偏置;
2  $p_d \leftarrow p + a$  // 计算偏置后的采样坐标;
3 return  $p_d$ 

```

图 5-29 动态云层模拟伪代码

其中, 使用归一化高度值 h 来调整不同海拔高度的云的运动速度, 使用时间 t 实时偏置采样坐标 p_d 的值, 从而实现不同海拔高速的云随时间运动的效果。

5.4.4 体积云的光照计算

完成对体积云的密度采样后,就需要对体积云进行光照计算,得到最终的颜色。体积云的光照模型仍旧满足体渲染方程,但由于体积云的密度特征与空气的密度特征不同,难以利用非实时阶段的预计算进行简化,因此在实时渲染阶段,若要对体积云的体渲染方程进行求解,就需要引入一些非物理的近似处理。本节将体积云的体渲染方程分解为直接光照,环境光照以及间接光照三个部分,分别进行计算,达到可信的云层着色效果。

要计算体积云的直接光照,就需要获取直接光照在到达光线步进采样点之前经历的能量变化,因此就需要在光线步进的每个采样点处,向光源方向进行步进采样,计算云层密度的路径积分。由于每次采样都对应了多次的纹理采样,为了平衡计算的开销和最终的着色效果,向光源方向采样的次数被限定在 6 次。并且由于云层内部存在大量的内散射,为了得到采样周围的微粒对采样点光照的影响,采样并不沿着直线进行,在朝向光源方向的圆锥内进行,具体的计算伪代码如图 5-30 所示:

Algorithm: 光路采样算法

Input: 起始坐标 p , 光源方向 $\vec{s}$, 步进步长 a , 吸收系数 A

Output: 光路方向上的光学深度 d_l

```

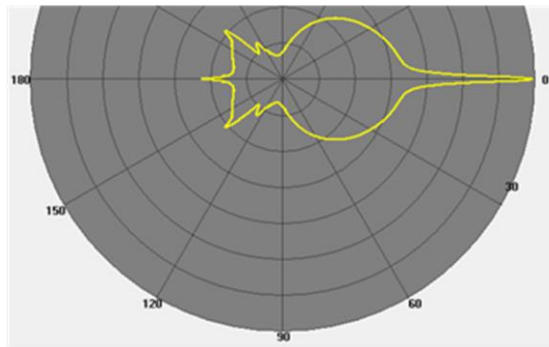
1  $R[6] \leftarrow$  初始化随机向量;
2  $i \leftarrow 0$ ;
3  $d_l \leftarrow 0$ ;
4 for  $i < 6$  do
5    $p \leftarrow p + R[i] \times i \times a \times \vec{s}$  // 计算采样坐标;
6    $h \leftarrow H(p.y)$  // 将采样坐标的高度值归一化;
7   if  $h \geq 0$  then
8      $d_l \leftarrow d_l + D(p)$  // 如果采样坐标在云层的高度范围内, 则采样云层的密度;
9   end
10 end
11  $d_l = d_l \times a \times A$  // 计算光学深度;
12 return  $d_l$ 
```

图 5-30 光路采样伪代码

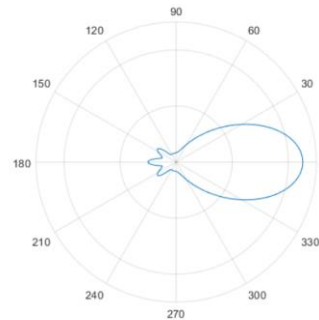
其中, $R[6]$ 中的随机向量均分布在圆锥的范围内, $H(p)$ 为前文所述的归一化高度值计算, $D(p)$ 为体积云密度采样函数。该算法返回的是光路方向上的光学深度。

有了光路方向上的光学深度之后,需要根据云的散射相函数计算经过散射进入视线方向的能量比例。由于米氏散射的散射相函数较为复杂,如图 5-31a 所示^[2],在前向和

后向上都有较强的散射, HG 散射公式很难对这样的散射进行近似, 因此引入一个基于插值算法的米氏散射相函数进行计算^[47]。该函数的图像如图 5-31b 所示



a) 云的米氏散射相函数



b) 本文使用的近似函数

图 5-31 云的米氏散射相函数与本文使用的近似函数的角度分布

在有了散射相函数之后, 就可以将视线与光线方向之间夹角的余弦值代入函数进行计算, 得到光线经散射进入视线的比例。如图 5-32 所示, 图 a 所显示的是未使用散射相函数的渲染结果, 图 b 展示的是使用了散射相函数的渲染结果。可以观察到在太阳附近, 使用了散射相函数的结果有着更高的亮度, 可以反映云层强烈的前向散射的特征。



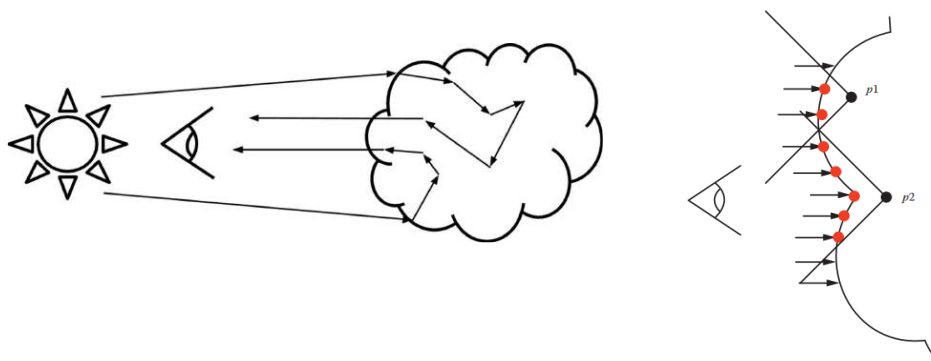
a) 未使用散射相函数的渲染结果



b) 使用散射相函数的渲染结果

图 5-32 未使用散射相函数的云层与使用散射相函数的云层

由于云层内部有着大量的外散射与内散射, 且这些散射若直接求解数值解, 计算将变得十分复杂, 因此本文使用 Andrew Schneider 提出的概率模型对这些最终的散射结果进行估计。

图 5-33 内散射概率示意图^[14]

如图 5-33 所示，云层最终渲染的颜色取决于光源发出的光线在云层内部经历一系列散射后，最终出射云层并进入视点的能量总额。如果考察云层内部靠近云层表面的点，这些点相交于云层表面的点，有更大的概率接受来自云层内部的散射光，并将这些光透过云层进入视点。随着与云层表面距离的增加，采样点收到散射光的概率也在增加，但是透射云层时的能量损失也在一并增加，因此可以用两个指数函数大致地描述这个过程，如公式（5-9）所示：

$$\begin{cases} p_{in}(d) = 1 - \exp(-2d) \\ p_t(d) = \exp(-d) \end{cases} \quad (5-9)$$

其中 $p_{in}(d)$ 和 $p_t(d)$ 分别描述了距表面距离为 d 的点收到的内散射强度概率和最终能够透过云层达到云层表面的能量比。将这个两个式子相乘并归一化，就能得到直接光照经过散射后进入视线方向的比例。如公式（5-10）所示。

$$E = 2 \exp(-d)(1 - \exp(-2d)) \quad (5-10)$$

这个函数的图像如图 5-34 所示，是一个先增大后减小的图像，可以反映云层散射强度的规律。

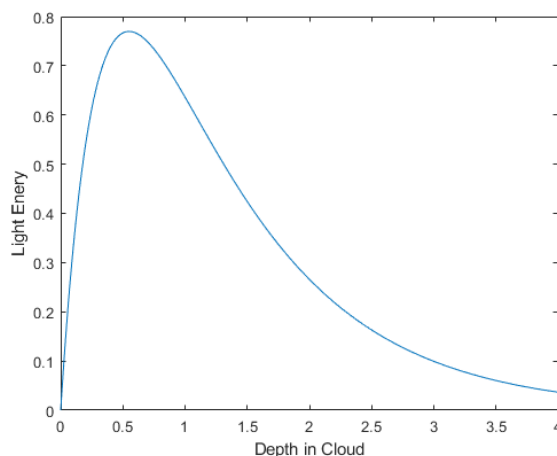


图 5-34 云层内不同深度的点出射光源能量比例示意图

未进行概率估计的云层渲染结果如图 5-35a 所示, 进行概率估计的云层渲染结果如图 5-35b 所示, 可以看到, 经过概率估计的渲染结果具有更多的明暗层次变化, 使得云层看起来更富立体感, 具有更好的渲染效果。



a) 未经概率估计的渲染结果

b) 概率估计后的渲染结果

图 5-35 未经概率估计的云层渲染与经过概率估计的云层渲染结果对比

在得到了光照的计算方法后, 就需要计算入射光的辐照度。入射的太阳光的辐照度可以使用光源方向上采样的终点坐标, 在 5.3.1 节介绍的预计算查找表中进行查找得到。有了太阳光照后还需要考虑环境光对云层的影响。对于云层而言, 环境光主要由两个部分组成, 来自天空的光线和来自地面的漫反射照亮云层底部的光线。来自天空的光线可以由像素对应的天空颜色代替, 计算的方法由 5.3.3 节给出, 来自地面的漫反射光会根据场景的变化而发生变化, 因此引入一个颜色变量来进行控制, 并且使用归一化的高度值将其进行缩放, 使得其强度随着云层高度的变化而发生变化。由于来自地面的漫反射光照强度与太阳的光照强度成大致正比, 因此可以用太阳光的强度对漫反射光强度进行控制, 达到环境光照随着太阳高度变化的效果。最终的环境光强度由来自天空的辐照度和地面漫反射辐照度插值得到, 具体的计算方式由公式 (5-11) 给出:

$$L_a = L_{bg} \times 0.2 + L_{bottom} \times (1 - h) \times A \times |L_{sun}| \quad (5-11)$$

没有考虑环境光照的云层与考虑了环境光照的云层渲染效果如图 5-36 所示, 可以看到若不考虑环境光照的影响, 云层的底部颜色会过于暗, 不符合实际情况。



a) 未考虑环境光照的渲染结果

b) 考虑环境光照的渲染结果

图 5-36 未考虑环境光与考虑环境光的云层渲染对比图

至此,本文计算了太阳入射光以及环境光对云层的直接光照,但此时的云层渲染结果仍旧过于暗淡,如图 5-37a 所示。为了计算多次散射引起的间接光照,本文使用了 Magnus Wrenninge 等人提出的多次散射近似计算方法,如公式(5-12)所示^[48]。

$$L = \sum_{i=0}^{N-1} L_i \quad (5-12)$$

$$L_i = \sigma_s b^i L_{light}(\omega_i) p(\cos^i \theta) e^{a^i \int_0^t \sigma_t(s) ds}$$

为了减少计算的开销,实际计算时,令 $N = 3$,可以得到较好的效果,如图 5-37b 所示。



a) 未考虑多次散射的渲染结果



b) 考虑多次散射的散射结果

图 5-37 未考虑多次散射与考虑多次散射的云层渲染对比图

至此,体积云的渲染基本完成,最终得到的云层颜色会随着太阳高度角的变化而产生明暗和颜色的变化,达到与实际情况近似的效果。最终的效果图如图 5-38 所示。



图 5-38 体积云渲染效果图

5.4.5 体积云渲染的优化

由于渲染体积云时,每个像素往往涉及多次纹理的采样,而纹理采样由于需要从显存中读取数据,受显存带宽的影响较大,往往被认为是图形渲染中最慢的环节,如果不加以优化,该算法的帧生成时间在 1080p 分辨率下,使用主流图形处理器,大约在 20ms 左右,难以满足实时渲染的需求。因此需要对算法进行优化。

经过分析可知，算法的主要瓶颈在于带宽的压力较大，因此要优化算法就需要从减少纹理采样次数的方式入手。本文使用了一种与时间反走样原理类似的优化算法，算法的示意图如图 5-39。

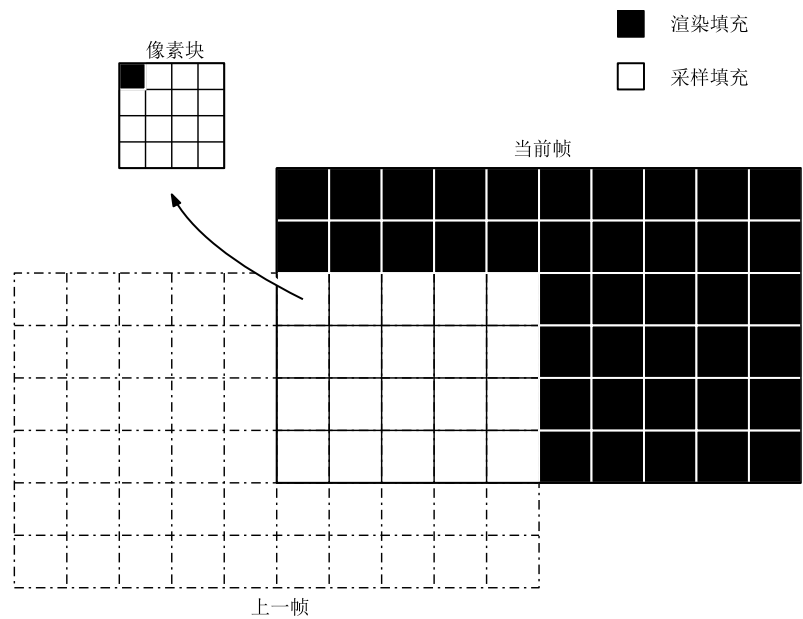


图 5-39 优化算法示意图

图中，虚线框表示上一帧的帧缓存，实线框表示的为当前帧的帧缓存，算法将帧缓存划分为多个 4×4 大小的像素块，对于每个当前帧的像素块，计算该像素块在上一帧的帧缓存中的位置。对于在上一帧的帧缓存范围内的像素块，当前帧只更新其中的一个像素，剩余的 15 个像素均从上一帧中直接采样获得。由于相机的移动，当前帧中可能存在一些像素对应的物体在上一帧中并没有被渲染，对于这些像素则全部进行渲染。完成渲染后，对上一帧的帧缓存进行更新。算法的伪代码如下：

Algorithm: 体积云优化算法

Input: 帧序号 i , 上一帧帧缓存 F_{i-1} , 上一帧相机变换矩阵 M_{i-1} , 当前帧相机逆变换矩阵 M_i^{-1} , 像素坐标 uv

Output: 像素的颜色 c

1

$uv_{i-1} \leftarrow M_{i-1}M_i^{-1}uv;$

2

$i \leftarrow \text{mod}(i, 16);$

3

if 像素在块内的序号 $= i \parallel (1, 1) < uv_{i-1}, uv_{i-1} < (0, 0)$ then

4

$c \leftarrow \text{RenderCloud}(uv)$ // 如果需要更新，则计算云层的颜色;

5

else

6

$c \leftarrow F_{i-1}(uv_{i-1})$ // 如果不需要更新，则从历史帧缓存中读取;

7

end

8

return c

图 5-40 体积云优化算法伪代码

需要注意的是,对上一帧的帧缓存进行采样时,所使用的采样器的过滤器必须是线性过滤器,如果使用邻近过滤器,则会在视角移动时出现明显的画面割裂,如图 5-41a 所示。使用线性过滤器后就能在视角变化时得到正确的结果,如图 5-41b 所示。



a) 使用邻近过滤器的渲染结果

b)使用线性过滤器的渲染结果

图 5-41 使用邻近过滤器生成的图像与使用线性过滤器生成的图像对比

同时,为了避免在视角变化时,帧缓存的边角上出现如图 5-42 所示的伪影,采样器的寻址模式应该设置为钳位寻址模式(Clamp)。



图 5-42 使用非钳位寻址的采样器产生的伪影

使用该优化算法后,由于帧缓存大约每 16 帧才完整地更新一次,因此极大地减小了图形处理器的带宽压力,经过测试,在 1080p 主流图形处理器的条件下,体积云的帧生成时间大约在 3.5ms 左右,可以满足实时渲染的需求。并且由于云层本身不具有规律的形状,不论是在静止条件下还是在云层,视角保持变化的条件下,该算法带来的渲染质量损失都难以被察觉。

5.5 渲染系统测试与性能分析

本节的内容主要包括对测试环境的介绍,系统的渲染效果以及系统的运行效率。

5.5.1 测试环境

表 5-1 列出了测试平台所使用的配置,本渲染系统的主要计算消耗在 GPU 上,而决定 GPU 性能表现的主要参数为 GPU 的显存速度,SM 单元的个数以及时钟频率。

表 5-1 测试平台配置

配置项	测试平台配置
CPU	i7-9750H 2.60GHz
内存	8GB*2 DDR4-2667MHz
GPU	NVIDIA GeForce GTX 1660Ti 6GB 显存 288GB/s 48 个 SM 单元 1500MHz

测试的主要方式为，运行程序，等待程序的帧率稳定之后，使用 RenderDoc 软件抓取一帧的数据，记录每个步骤的耗时。

5.5.2 大气渲染的效果及效率

图 5-43 和图 5-44 分别展示了在不同时间段（白天，黄昏）时，场景中的物体在环境光照的影响下呈现出的不同颜色，以及远处地面在光线透射的影响下呈现的雾态效果。

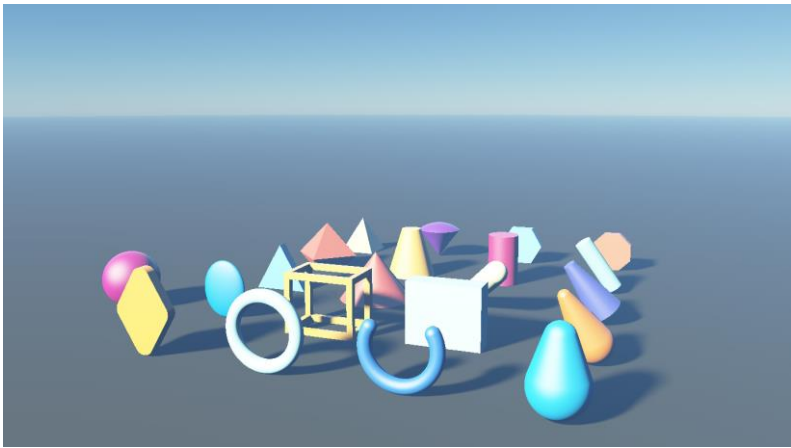


图 5-43 白天时物体的着色

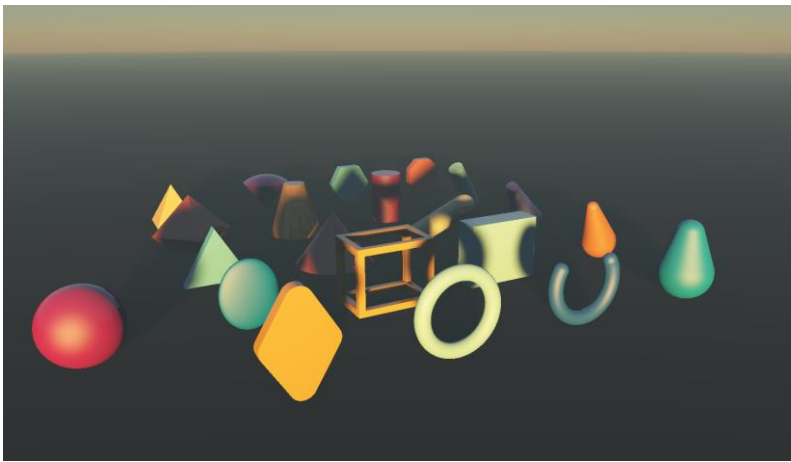


图 5-44 黄昏时物体的着色

图 5-45 和图 5-46 分别呈现了在不同的时间段时，天空和地面的着色效果。



图 5-45 白天时天空与地面的着色效果

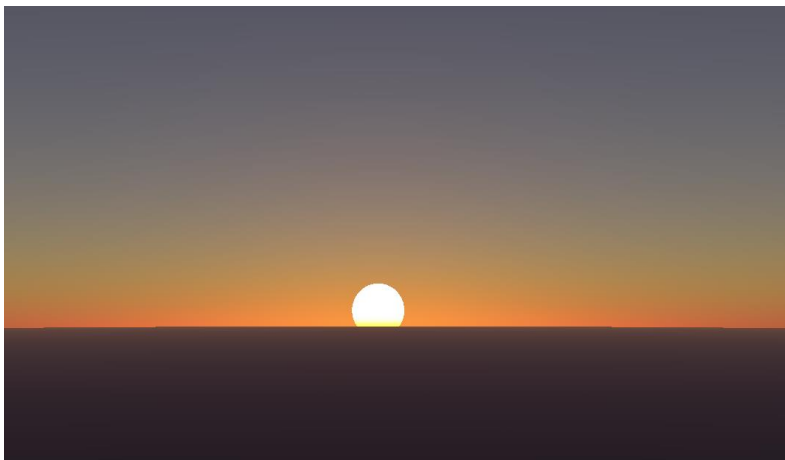
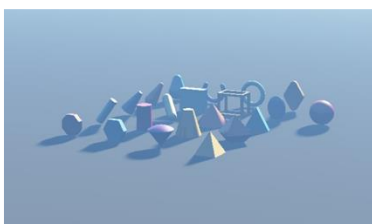
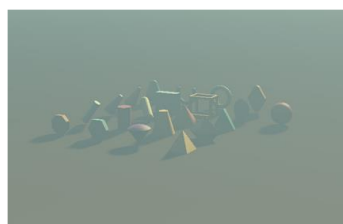


图 5-46 黄昏时天空与地面的着色效果

由于空气中的污染物大多发生米氏散射，通过对大气层中米氏散射强度的调整，可以对不同污染程度的大气进行模拟，图 5-47abc 分别展示了无污染条件下和不同污染程度下的大气效果。



a) 无污染情况下



b) 污染程度 A



c) 污染程度 B

图 5-47 无污染情况与不同污染程度下的大气效果

图 5-48abc 分别展示了从不同海拔高度以及从太空中观察天空和地面的渲染效果。可以看到随着海拔高度的提升，可以看到地平线的变化，以及地球的晨昏线。



a) 海拔高度 10000 米

b) 海拔高度 50000 米

c) 从太空中观察地球

5-48 不同海拔高度下的渲染结果

在只进行大气渲染时，在 1080p 的分辨率下，帧生成时间大约在 0.95ms 左右，能够满足实时渲染的需求。

5.5.3 增加云层渲染后的效果及效率

影响体积云的渲染效率的主要因素有：天空中云层的覆盖比例，光线步进次数，以及渲染的分辨率。其中，光线步进次数和渲染的分辨率还会对渲染的质量产生影响。因此首先测试天空中云层的覆盖比例对渲染效率的影响。通常而言，云层覆盖的比例越高，需要计算的部分也就越多，帧生成时间也就越长。通过调整云层的在天空中的占比，对帧生成时间进行测试，在 1080p 渲染分辨率下，具体的帧生成时间如图 5-49 所示。

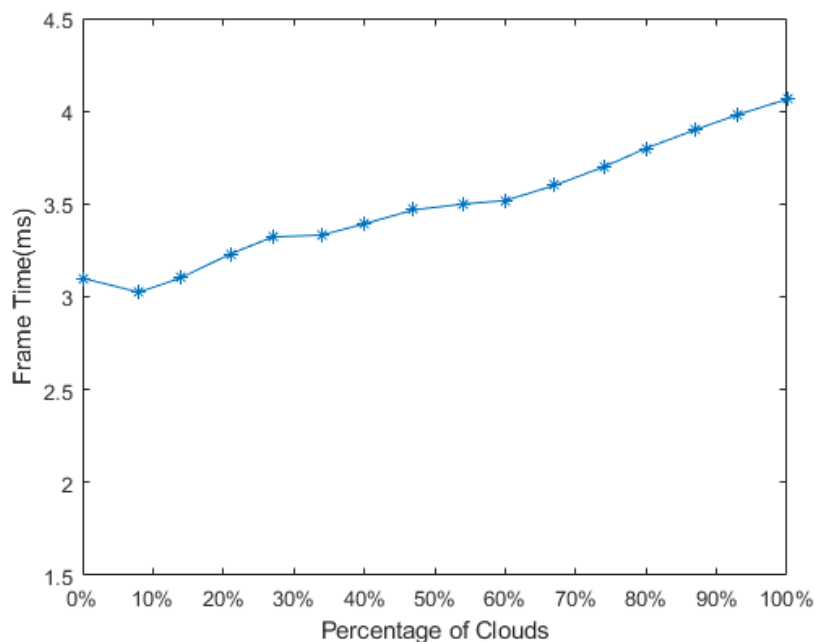


图 5-49 不同云层覆盖比率例的帧生成时间折线图

可以观察到，云层的覆盖率越高，帧生成时间也随之变长。

接下来测试光线步进次数对体积云渲染效果的影响。光线步进次数约多，效果越好，同时效率也会降低。不同步进次数时的效果分别如图 5-50，图 5-51，以及图 5-

52 所示:



a) 光线步进次数 96-128 次



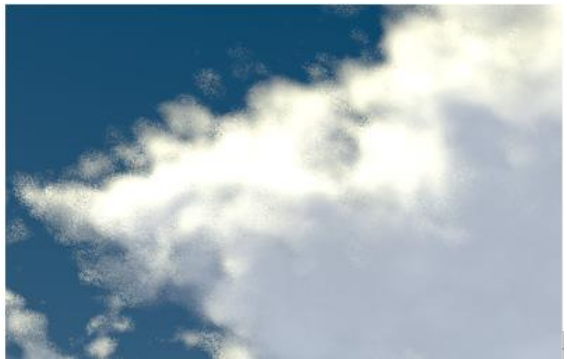
b) 光线步进次数 64-128 次

图 5-50 光线步进次数较多时的效果

图 5-50 展示的是光线步进次数较多时的云层渲染效果。可以看到两者之间的效果差异并不大, 并且云层中的噪点比较少。



a) 光线步进次数 64-96 次



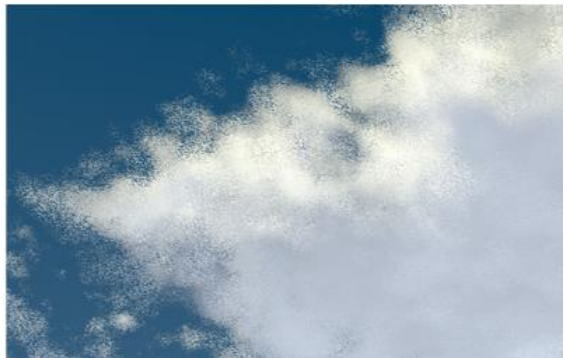
b) 光线步进次数 32-96 次

图 5-51 光线步进次数中等时的渲染效果

图 5-51 展示的是光线步进次数中等时, 云层的渲染效果。可以观察到步进次数在 64-96 次时, 云层的渲染效果与步进次数较多时的相似; 步进次数调整到 32-96 次之后, 已经可以观察到有噪点产生。



a) 光线步进次数 32-64 次

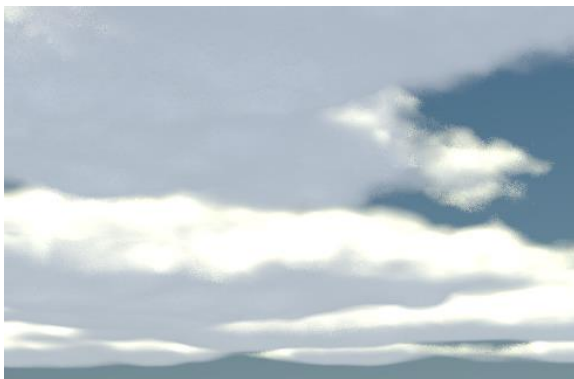


b) 光线步进次数 16-32 次

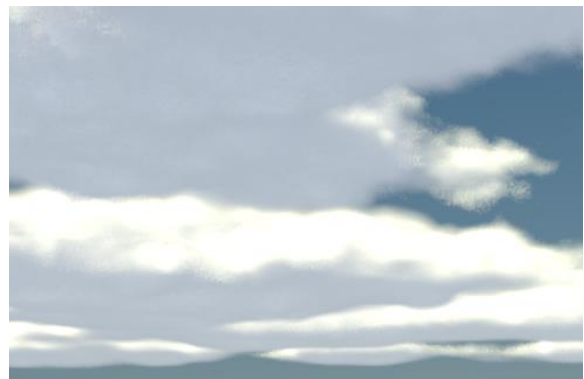
图 5-52 光线步进次数较少时的渲染效果

图 5-52 展示的是光线步进次数较少时，云层的渲染效果。可以观察到随着步进次数的继续减少，噪点会明显增加。步进次数减少到 16-32 次时，渲染的质量已经有了明显的下降。

云层的渲染效果与效率同渲染所使用的分辨率也有很高的关联性。现选取光线步进次数为 96-128，64-96，以及 32-64 次的三种模式，分别对其进行 720p 的渲染后，线性插值到 1080p 的帧缓存上，比较两者之间的渲染效果。对比图如图 5-53，图 5-54，图 5-55 所示。



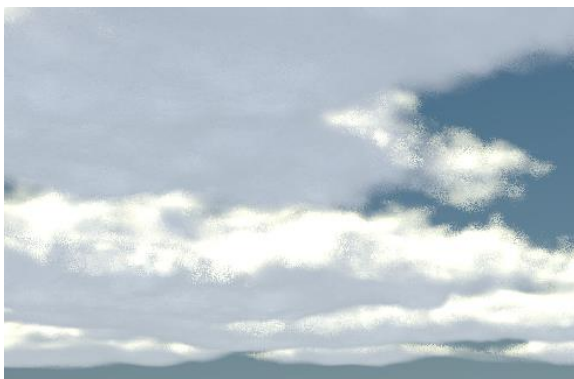
a) 1080p 渲染



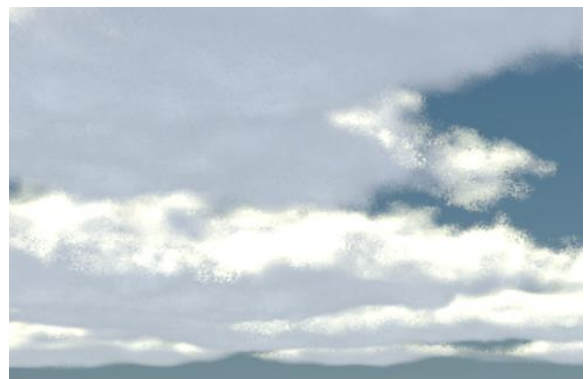
b) 720p 渲染

图 5-53 光线步进次数 96-128 次时 1080p 与 720p 渲染效果对比

图 5-53 展示的是当步进次数较多时，1080p 与 720p 的渲染效果之间的对比。可以观察到两者之间几乎没有差别。



a) 1080p 渲染



b) 720p 渲染

图 5-54 光线步进次数 64-96 次时 1080p 与 720p 渲染效果对比

图 5-54 展示的是当步进次数中等时，1080p 与 720p 的渲染效果之间的对比。可以观察到，在有噪点产生之后，720p 的渲染效果较 1080p 有略微的模糊，但不影响整体的渲染质量。

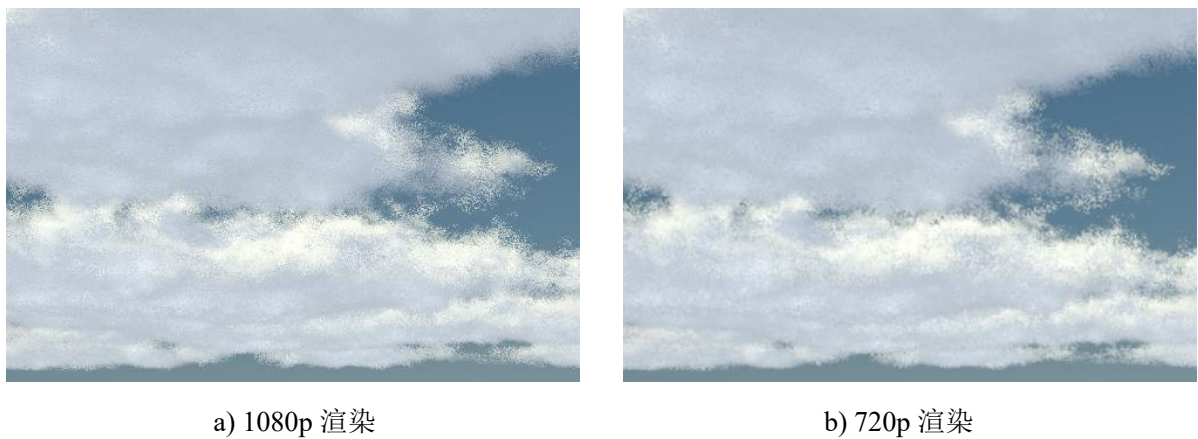


图 5-55 光线步进次数 32-64 次时 1080p 与 720p 渲染效果对比

图 5-55 展示的是当步进次数较少时，1080p 渲染与 720p 渲染效果之间的对比。可以观察到，尽管 720p 相较于 1080p 有了更为明显的模糊，但影响渲染质量的主要还是步进次数过少引起的噪声。

表 5-2 记录了不同步进次数，不同渲染分辨率，以及关闭体积云渲染优化等条件下的帧生成时间。

表 5-2 不同渲染条件下的帧生成时间对比表

步进次数	1080p(ms)	720p(ms)	1080p 无优化(ms)	720p 无优化(ms)	1080p 优化幅度	720p 优化幅度
96-128	5.19	2.47	37.72	16.88	727%	683%
64-128	4.39	2.09	31.36	13.9	714%	665%
64-96	4.28	2.01	27.08	12.05	633%	600%
32-96	3.83	1.7	21.43	9.78	560%	575%
32-64	3.53	1.681	17.36	8.23	492%	490%
16-32	3.21	1.42	13.24	6.73	412%	474%

通过综合考虑渲染效果与渲染效率，可以选用光线步进次数中等，渲染分辨率 720p 的渲染方案。此时帧生成时间在 1.7ms 到 2ms 之间，能够满足实时渲染的需求。图 5-56 和图 5-57 展示了在该渲染条件下，不同时间段时，系统的整体渲染效果，可以观察到环境光照对云层着色的影响，能够较好地还原真实的场景。



图 5-56 白天时云层的着色



图 5-57 黄昏时云层的着色

5.5.4 与其他方法的比较

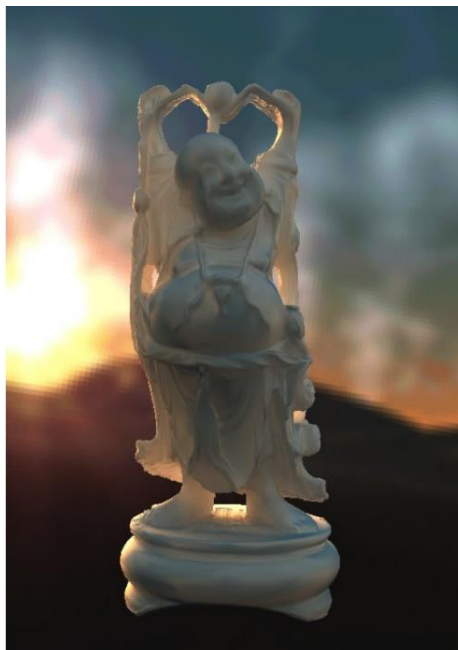
传统的渲染天空的方法主要为立方体映射或球面映射，具体的方法是，将预先制作好的纹理，通过纹理映射的方式，将纹理的颜色映射到一个立方体或者球的内部，得到环境的渲染结果。之后在这个球的内部进行场景物体的渲染。如果要表现环境光照对物体的影响，广泛应用的有两种方法：基于图片的光照(Image Based Lighting)，以及球谐光照(Spherical Harmonic Lighting)方法。这两种也都是基于预计算的方法。现对两种方法进行简单介绍。

基于图片的光照会对天空纹理进行预处理，对于不同粗糙度系数，对天空纹理进行卷积，并记录多个查找表。同时，关于渲染方程中的物体材质参数，对于所有可能的取值，建立一个二维的查找表。并在实时渲染阶段查找临近结果，并进行插值得到近似结果。图 5-58 展示了基于图片的光照的渲染结果。

图 5-58 基于图片的光照的渲染结果^[49]

对于基于图片的光照而言，渲染质量较高，渲染速度也较快，场景中的物体可以进行移动，但并不支持动态的环境光变化，也不能渲染动态的云层，同时也不支持环境光阴影。

球谐光照则使用三维的球谐函数对场景中的静态光源以及静态物体进行预计算，得到各个球谐函数的系数，然后在实时阶段，利用这些系数对场景中的光照信息进行重建，类似于对函数进行傅里叶展开后，利用傅里叶函数的前若干项还原函数。球谐光照的渲染结果如图 5-59 所示。

图 5-59 球谐光照的渲染结果^[50]

对于球谐光照而言，进行渲染的场景必须是静态的，同时，渲染的质量取决于预计算时的计算量，如果预计算时使用的球谐函数阶数过少，则在最终的渲染结果中会丢失许多高频的细节。并且球谐光照也不支持动态的场景。但球谐光照渲染速度较快，且

支持环境光阴影。

目前，在计算机图形学界的实时渲染领域，衡量一种方法主要有两个维度，效果和效率。对于效果的衡量又可以从功能方面分为多个维度，比如是否支持环境光照的计算，是否支持环境光照阴影的计算，是否支持场景中的物体的移动等等。由于不同方法之间的具体路径不同，因此不同方法可能在功能维度上有较大的区别；而又因为实时渲染领域的方法都是对渲染方程的简化，计算渲染方程的近似解，指导思想相同，因此对于支持相同功能的两种不同算法，在最后呈现的效果上的差别不大，难以做定量的比较，因此本文认为对不同方法做定性比较较为合理。

在计算效率方面，不同算法所需的数据完全不同，难以控制变量进行准确的比较，与本文相似的方法也难以找到公开，可复现的实现，因此在计算效率方面，本文也只根据实际的应用场景进行了定性比较。表 5-3 总结了上述方案与本文提出的方案之间的区别：

表 5-3 本文所述方案与 IBL 和 SHL 的对比

渲染方案	是否预计算	预计算时长	动态场景	动态环境	散射效果	动态云层	环境光阴影	渲染速度
IBL	是	短	支持	不支持	无	不支持	不支持	较快
SHL	是	长	不支持	仅支持旋转	无	不支持	支持	快
本文	是	短	支持	支持	有	支持	支持	中

通过对比可得，尽管在渲染的速度上，本文的方法逊于上述两种常用的方法，但本文的方法解决了上述方法无法实现动态环境光照以及动态云层的问题。并且在预计算时长以及是否支持环境光阴影等方面，相较于前两种方法各有优势。

5.6 本章小结

本章主要阐述了基于物理的大气实时渲染技术的系统结构和实现细节。首先对渲染系统的总体架构以及渲染的具体流程进行了介绍。其次，根据本文所实现的具体程序，对大气渲染过程中的各个预计算项的计算流程进行了给出了实现的伪代码，并且介绍了提升在地平线附近的精度的存储方式。之后对云层的渲染给出了具体的流程，并针对云层渲染中可能出现的走样给出了解决的方案。同时，给出了云层渲染的优化方法，提升了渲染的效率。最后搭建了测试环境，展示了系统的整体效果以及性能表现，并且将本文的方法与常用的环境光照算法进行对比。

总结与展望

本文对地球的大气以及云层进行了建模,实现了相应的渲染算法,提升了室外场景的渲染结果的真实感。本文从以下几个方面对所做的工作进行了介绍。首先介绍了基于物理的渲染的理论基础:计算机色彩理论以及辐射度量学,这两者共同给出了较为准确地进行光照计算的方法。并且由此介绍了体渲染的原理,以及需要求解的体渲染方程。

在有了体渲染的理论之后,本文对大气的渲染的方法进行了介绍,对大气渲染中,体渲染方程的各项进行了分析,针对大气分布的特点,对使用预渲染算法对大气渲染进行加速的方法进行了介绍,并对大气的透射率,单次散射辐射率,多次散射辐射率的计算方法进行了介绍。

在介绍了大气渲染的方法之后,本文对云层的特性进行了研究,介绍了使用噪声对云层形状以及分布进行模拟的方法,并介绍了云层的光照模型以及优化体积云渲染效率的方法。

最后,本文对实现的渲染系统的整体架构进行了介绍,利用 DirectX 12 的特点,设计了渲染的方法,并对本文之前介绍的各个部分的实现进行了具体的介绍,给出了相关部分的伪代码。并将大气效果与体积云效果进行了整合,得到了较为完整的,可以随着时间变化而改变着色效果的云层和大气层,得到了较为逼真的视觉效果。并且经过测试,算法可以满足实时渲染的效率需求。

在实现了以上的内容后,本文实现的渲染算法仍旧有比较大的改进空间。首先是大气渲染,其中对于预计算纹理的使用仍旧有优化的空间,可以通过对算法的优化,减小预计算纹理的存储空间大小,以及实时渲染时采样的次数,从而提升算法的效率。其次,本文没有实现由于物体对光线的遮挡产生的体积阴影的渲染,在视觉效果上有提升的余地。在云层的渲染部分,出于性能的考虑,并没有实现体积云的阴影,可以使用相应的体积材质阴影映射技术实现体积云的阴影,提升算法的视觉效果。另外,由于复用了之前帧的渲染结果,会在与场景混合渲染时,渲染结果会跟随相机视角产生拖影。可以考虑使用 TAA 减少拖影的技术减少拖影的产生。

致 谢

三年的硕士研究生时光很快过去，回顾研究生阶段的三年，更多的是收获和喜悦。这三年里，身边的老师，朋友都给了我莫大的帮助，在此表示感谢。

首先需要感谢的是我的导师，苏虎副教授，感谢苏老师为我在研究生阶段提供的优良的研究环境，以及在学习和生活方面提供的帮助，让我可以心无旁骛地在自己感兴趣的领域进行钻研，并取得长足的进步。并且，苏老师还提供了大量的科研项目，让我在其中锻炼了工程能力。我在研究生阶段取得的进步离不开苏老师的大力支持，感谢苏老师的悉心栽培，愿苏老师事业蒸蒸日上，桃李满天下。

还要感谢的是陈宏川师兄。在我本科以及研究生阶段，我在陈师兄的指导下工作了四年左右，陈师兄无私地传授了他多年的知识积累和经验，在我遇到问题时也对我进行了耐心的指导。非常感谢陈师兄的付出，愿师兄生活一帆风顺。

也感谢我的父母，对我多年来的关怀和培养，来自家庭的支持永远是最好的支持。

需要感谢的还有我的女朋友，白雅婷同学，感谢与她相识的六年多时间里，对我学业和生活上的支持。还有我的朋友丁冠期同学，楼宇迪同学，周琳同学，以及我的师兄、师弟、师妹们，感谢你们多年来的陪伴。

最后，感谢抽出宝贵时间审阅这篇论文的老师，谢谢你们付出！

参考文献

- [1] Nishita T, Sirai T, Tadamura K, et al. Display of the earth taking into account atmospheric scattering[C]. Proceedings of the 20th annual conference on Computer graphics and interactive techniques. 1993: 175-182.
- [2] Hillaire S. Physically based sky, atmosphere and cloud rendering in Frostbite[C]. ACM SIGGRAPH. 2016: 1-62.
- [3] Preetham A J, Shirley P, Smits B. A practical analytic model for daylight[C]. Proceedings of the 26th annual conference on Computer graphics and interactive techniques. 1999: 91-100.
- [4] Akenine-Möller T, Haines E, Hoffman N. Real-time rendering[M]. 4th ed. Crc Press, 2019.
- [5] O'Neil, Sean. Accurate Atmospheric Scattering[M]//Pharr M. GPU Gems 2: Programming techniques for high-performance graphics and general-purpose computation. Addison-Wesley, 2005: 253-268.
- [6] Bruneton E, Neyret F. Precomputed atmospheric scattering[C]. Computer graphics forum. Oxford, UK: Blackwell Publishing Ltd, 2008, 27(4): 1079-1086.
- [7] Elek O. Rendering parametrizable planetary atmospheres with multiple scattering in real-time[C]. Proceedings of the Central European Seminar on Computer Graphics. Citeseer, 2009.
- [8] Yusov E. Outdoor Light Scattering[EB/OL].(2013-06-26)[2021-01-20]. <https://software.intel.com/content/www/us/en/develop/blogs/outdoor-light-scattering-sample.html>.
- [9] Hillaire S. A Scalable and Production Ready Sky and Atmosphere Rendering Technique[C]. Computer Graphics Forum. 2020, 39(4): 13-22.
- [10] Guerrette K. Moving The Heavens[EB/OL].(2014-03-19)[2021-01-20]. <https://www.gdc-vault.com/play/1020474/Moving-the-Heavens-An-Artistic>.
- [11] Yusov E. High-Performance Rendering of Realistic Cumulus Clouds Using Pre-computed Lighting[C]. High Performance Graphics. 2014: 127-136.
- [12] Bouthors A, Neyret F, Max N, et al. Interactive multiple anisotropic scattering in clouds[C]. Proceedings of the 2008 symposium on Interactive 3D graphics and games. 2008: 173-182.
- [13] Fong J, Wrenninge M, Kulla C, et al. Production volume rendering: SIGGRAPH 2017 course[C]. ACM SIGGRAPH 2017 Courses. 2017: 1-79.
- [14] Schneider A, Engel W. Real-time volumetric cloudscape[J]. GPU Pro 7: Advanced

- Rendering Techniques, 2016: 97.
- [15] Bauer F. Creating the Atmospheric World of Red Dead Redemption 2: A Complete and Integrated Solution: SIGGRAPH 2019 course[C]. ACM SIGGRAPH 2017 Courses. 2019:1-79.
- [16] Fairchild M D. Color appearance models[M]. 2nd ed. John Wiley & Sons, 2013.
- [17] Grassmann H. Zur theorie der farbenmischung[J]. Annalen der Physik, 1853, 165(5): 69-84.
- [18] Stokes M, Anderson M, Chandrasekar S, et al. Standard default color space for the Internet[J]. Tech. Rep., World Wide Web Consortium, 1996.
- [19] Drigo A. Fundamental University Physics — Vol. III: Quantum and Statistical Physics[J]. Nuovo Cimento A Serie, 1970, 67(3): 574-575.
- [20] Pharr M, Jakob W, Humphreys G. Physically based rendering: From theory to implementation[M]. Morgan Kaufmann, 2016.
- [21] Karttunen H, Kröger P, Oja H, Poutanen M, Donner K J. Fundamental astronomy[M]. Springer, 2016.
- [22] West W. Absorption of electromagnetic radiation[J]. AccessScience, McGraw-Hill Companies, 2008.
- [23] Jarosz W. Efficient Monte Carlo methods for light transport in scattering media[D]. University Of California, San Diego, 2008.
- [24] Ingle Jr J D, Crouch S R. Spectrochemical analysis[J]. 1988.
- [25] Henyey L G, Greenstein J L. Diffuse radiation in the galaxy[J]. The Astrophysical Journal, 1941, 93: 70-83.
- [26] Chandrasekhar S. Radiative transfer[M]. Courier Corporation, 2013:66-69.
- [27] Hayes D S, Latham D W. A rediscussion of the atmospheric extinction and the absolute spectral-energy distribution of VEGA[J]. The Astrophysical Journal, 1975, 197: 593-601.
- [28] Rayleigh J W S B. On the scattering of light by small particles[J]. Philosophical Magazine, 1871, 61:447-454,64.
- [29] Hahn D W. Light scattering theory[J]. Department of Mechanical and Aerospace Engineering, University of Florida, 2009.
- [30] Jacob D J. Introduction to atmospheric chemistry[M]. Princeton University Press, 1999:10.
- [31] O3 Spectra[EB/OL].(2011-06-23)[2021-01-28]. <https://www.iup.uni-bremen.de/gruppen/molspec/databases/referencespectra/o3spectra2011/index.html>.
- [32] Cloud. Wikipedia[EB/OL].(2020-12-15)[2021-01-30]. <https://en.wikipedia.org/wiki/-Cloud>.

-
- [33]Perlin K. An image synthesizer[J]. ACM Siggraph Computer Graphics, 1985, 19(3): 287-296.
- [34]Ebert D S, Musgrave F K, Peachey D, et al. Texturing & modeling: a procedural approach[M].3rd ed. Morgan Kaufmann, 2003:72-77.
- [35]Worley S. A cellular texture basis function[C]. Proceedings of the 23rd annual conference on Computer graphics and interactive techniques. 1996: 291-294.
- [36]Vivo P G, Lowe J. Cellular Noise. The Book of Shaders[EB/OL].[2021-02-02]. <https://thebookofshaders.com/12/>.
- [37]Mandelbrot B B, Mandelbrot B B. The fractal geometry of nature[M]. New York: WH freeman, 1982.
- [38]Schneider A, Vos N. Nubis: Authoring Real-Time Volumetric Cloudscapes with the Decima Engine[C]. SIGGRAPH Advances in Real-Time Rendering in Games course,2017: 619-620.
- [39]Kevin. Colorado Clouds Blog[EB/OL].(2013-03-31)[2021-02-11]. <https://coclouds.com/478/stratus/2013-03-21/>.
- [40]Means T. The 10 Basic Types of Clouds[EB/OL].(2019-07-03)[2021-02-11]. <https://www.thoughtco.com/types-of-clouds-recognize-in-the-sky-4025569>.
- [41]Staughton J. What Is A Cumulonimbus Cloud?[EB/OL].(2019-12-05)[2021-02-11]. <https://www.scienceabc.com/nature/what-is-a-cumulonimbus-cloud.html>.
- [42]秦春林. 全局光照技术：从离线到实时渲染[M]. 成都：电子科技大学出版社, 2018.
- [43]GeForce 256: The World's First GPU. nVIDIA[EB/OL].(2021-01-19)[2021-02-20]. <https://web.archive.org/web/20060324123500/http://www.nvidia.com/page/geforce256.html>.
- [44]Luna F. Introduction to 3D game programming with DirectX 12[M]. Mercury Learning & Information, 2012.
- [45]Jacques. DirectX 12 and Fortnite[EB/OL].(2019-11-20)[2021-02-20]. <https://devblogs.microsoft.com/directx/directx12-and-fortnite/>.
- [46]Morgan T P. Diving Deep into the NVIDIA Ampere GPU Architecture[EB/OL].(2020-05-28)[2021-02-20]. <https://www.nextplatform.com/2020/05/28/diving-deep-into-the-nvidia-ampere-gpu-architecture/>.
- [47]Schander T. Cloud Phase function comparison. Shadertoy[EB/OL].(2017-08-05)[2021-02-25]. <https://www.shadertoy.com/view/4sjBDG>.
- [48]Wrenninge M, Kulla C D, Lundqvist V. Oz: the great and volumetric[C]//SIGGRAPH
-

Talks. 2013: 46:1.

[49]Joey V. Learn OpenGL: Learn modern OpenGL graphics programming in a step-by-step fashion[M].3rd ed. Kendall & Welling, 2020.

[50]Ramamoorthi R. Precomputation-based rendering[M]. NOW Publishers Inc, 2009.

攻读硕士学位期间发表的论文及科研成果

论文发表：

- [1] Chen K, Su H. A Motorcycle Driving Simulator Based on BikeSim[C]. Advanced Science and Industry Research Center.Proceedings of 2019 International Conference on Computational Modeling,Simulation and Optimization. 2019: 60-64.
- [2] Chen K, Su H, Cheng J. A Vehicle Driving Simulator Based on Mixed Reality[C]. CICTP 2020. 2020: 3948-3958.
- [3] Su H, Wang R, Chen K, et al. A Simulation Method for LIDAR of Autonomous Cars[C]. IOP Conference Series: Earth and Environmental Science. IOP Publishing, 2019, 234(1): 012055. (EI: 20191306697154)

科研项目：

- [1] 2018-2019. 香港警察交通学校 HKPF 冲锋车及警用摩托车仿真器研发项目.
- [2] 2019-2020. 混合现实仿真器技术研发项目. 2019H010967.

专利：

- [1] 苏虎，王蕊，陈凯鑫，田国庆. 一种基于几何体求交运算的车载激光雷达仿真方法. 国家发明专利. 专利号：ZL 2018 1 0942482.0. 已授权.